

Architecture Evolution of Google Guava

EPL484 Software Evolution – Project 1 – Team X

<Member 1 (ID)>, <Member 2 (ID)>

October 2026

- [1. Project choice and characteristics](#)
 - [1.1 What is Guava?](#)
 - [1.2 Eligibility \(Phase 1\)](#)
 - [1.3 Number and size of versions](#)
 - [1.4 Is the architecture documented?](#)
- [2. Methodology](#)
 - [2.1 Data collection](#)
 - [2.2 Data cleaning and version selection](#)
 - [2.3 Tools](#)
 - [2.4 Architecture recovery \(A1-A4\)](#)
 - [2.5 Evaluation](#)
- [3. Results and discussion](#)
 - [3.1 Size](#)
 - [3.2 Class dependencies](#)
 - [3.3 Widely-used \(noise\) classes](#)
 - [3.4 Recovered architectures \(A1-A4\)](#)
 - [3.5 Quality of the architecture over time](#)
 - [3.6 Comparison with the reference architecture \(MoJoFM\)](#)
 - [3.7 Widely-used classes: JNode vs Bunch](#)
 - [3.8 Architectural smells and their evolution](#)
 - [3.9 Lehman's laws: dependency changes and work rate](#)
- [4. Architecture recovery with AI \(latest version only: 33.7.2\)](#)
- [5. Reflection](#)
 - [5.1 Comparing the architectures of the latest version](#)
 - [5.2 Does the AI tool help? How do information and prompts influence the result?](#)
 - [5.3 Which method for other systems?](#)
- [6. Threats to validity](#)
- [References](#)
- [Appendix A – Full metric tables](#)
- [Appendix B – The AI prompts \(literal\)](#)

Code and data: GitHub repository `epl484.fall26.project1.teamX`, folder `guava/` (data in `guava/data/`, results in `guava/results/`, AI material in `guava/ai/`, report in `guava/report/`); the analysis scripts are in `scripts/`. Every table in this report is generated from the result files by `scripts/report_tables.py`.

1. Project choice and characteristics

1.1 What is Guava?

[Guava](#) is Google's set of core libraries for Java. It is used in most Java projects inside Google and in a large part of the Java ecosystem. It offers:

- **Collections** – immutable collections (`ImmutableList`, `ImmutableMap`, ...), new collection types (`Multimap`, `Multiset`, `BiMap`, `Table`), ranges (`Range`, `RangeSet`), and utilities (`Lists`, `Maps`, `Sets`,

Iterables, FluentIterable, Ordering);

- **Base utilities** – Preconditions, Optional, functional types, string handling (Joiner, Splitter, CharMatcher), Stopwatch;
- **Concurrency** – ListenableFuture and the Futures combinators, executors, Service, RateLimiter;
- **Caching** (CacheBuilder, LoadingCache), a **graph library** (Graph, ValueGraph, Network), **I/O** (ByteSource, CharSource, Files), **hashing** (Hashing, BloomFilter), **primitives and math**, **reflection** (TypeToken, ClassPath), an **event bus**, **escaping** and **networking** helpers (MediaType, InternetDomainName).

Guava is a **library**. It is not a running system with processes or deployed components; it has runtime behaviour (thread pools in `util.concurrent`, the cache, the event bus), but this study looks only at its **static** architecture: the decomposition into packages and classes and the dependencies between them, as extracted from the bytecode.

1.2 Eligibility (Phase 1)

Criterion	Guava
≥ 10 major/minor versions	54 major/minor releases (33 major v1.0–v33.0, 21 minor), plus 28 patch releases and 25 release candidates (109 v* tags)
Repository on GitHub	google/guava
Latest version ≥ 10K LOC	33.7.2: 611 production .java files, 96,646 non-comment, non-blank LOC
Not archived / not a fork	✓ / ✓
≥ 50 commits since 1/1/2024	1,261 (of 7,546)
Created before 2024	history starts 2009-06-18

1.3 Number and size of versions

We analyse a **sparse sample of 13 releases**: every second major release from 10.0 to 32.0 plus **33.7.2**, the latest release at the time of the analysis (29 Sep 2026; a patch release of the 33.7 feature release). This covers 15 years (September 2011 – September 2026), and no two sampled releases are consecutive. Table 1 gives the size of each version.

- *Class files* include inner and anonymous classes.
- *Top-level classes (jar)* are the compilation units in the jar.
- *In dependency data* counts the top-level classes that appear in the DependencyExtractor output.
- *Connected* counts the classes with at least one dependency to or from another Guava class. These are the classes that are clustered (A1, A3).
- *Dependencies* are distinct dependencies between top-level classes.

Table 1 – Size of the analysed versions.

Version	Date	Packages	Class files	Top-level classes (jar)	Δ classes	In dependency data	Connected	Dependencies	Deps / connected class
10.0	2011-09-27	10	1157	317	–	310	302	1289	4.27
12.0	2012-04-30	13	1332	373	+17.7%	364	354	1597	4.51
14.0	2013-02-25	13	1584	412	+10.5%	399	390	1945	4.99
16.0	2014-01-17	17	1663	448	+8.7%	435	425	2121	4.99
18.0	2014-08-25	17	1677	454	+1.3%	439	429	2112	4.92
20.0	2016-10-28	18	1799	514	+13.2%	499	491	2453	5.00
22.0	2017-05-22	18	1850	529	+2.9%	514	506	2534	5.01
24.0	2018-02-01	18	1922	549	+3.8%	534	527	2633	5.00
26.0	2018-08-01	18	1936	556	+1.3%	541	535	2686	5.02
28.0	2019-06-11	18	1932	554	-0.4%	539	533	2699	5.06
30.0	2020-10-16	18	2010	569	+2.7%	553	547	2769	5.06
32.0.0	2023-05-26	18	1997	607	+6.7%	558	553	2821	5.10
33.7.2	2026-09-29	18	1948	594	-2.1%	554	549	2750	5.01

1.4 Is the architecture documented?

Only at the API level. The README and the [user guide](#) describe the packages feature by feature (“Collections”, “Caches”, “Functional idioms”, “Concurrency”, “Strings”, “Primitives”, “Ranges”, “I/O”, “Hashing”, “EventBus”, “Math”, “Reflection”, “Graphs”), and the Javadoc documents each package. The wiki also documents policies with architectural consequences: a strict **API-compatibility** policy, where @Beta APIs may change and the rest are only removed after deprecation, and the separate **JRE and Android flavours**. There is **no component/dependency view** and no description of the internal structure of the large packages. The package structure *is* the documented architecture.

2. Methodology

The analysis scripts are in `scripts/`; the Guava-specific data collection is in `guava/scripts/`.

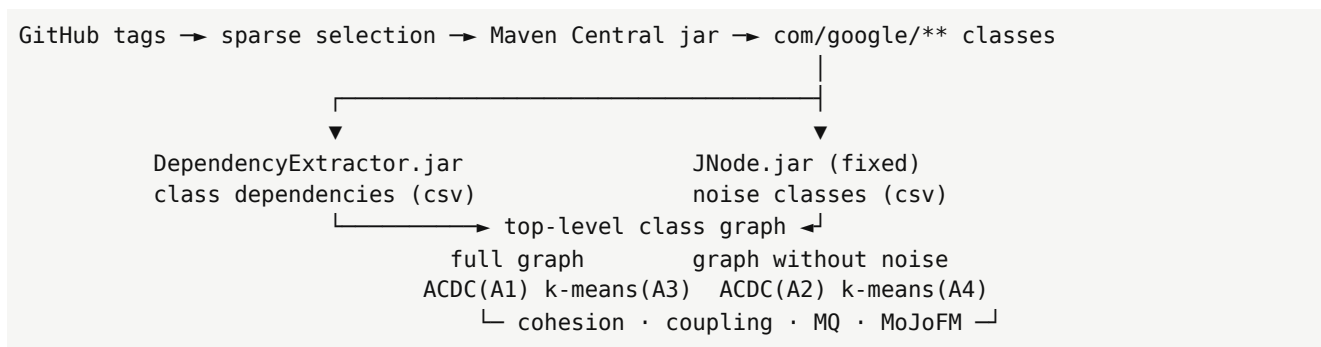


Figure 1 - Analysis pipeline.

2.1 Data collection

1. **Releases.** All tags were read from a blob-less clone of the repository with their creation dates (guava/data/tags.csv). The Phase 1 statistics come from the same clone (commit counts, NCLOC of guava/src at v33.7.2).
2. **Binaries.** Guava is a single Maven artifact (com.google.guava:guava). guava/scripts/collect_versions.py downloads the jar of every selected release from Maven Central. From 23.1 on Guava is published in two flavours (-jre, -android); we use the main -jre flavour. The script keeps the com/google/** classes (Guava's own code, including com.google.thirdparty.publicsuffix) and places the jar in guava/data/workspace/<v>/bin/, the input layout of the course tools.

2.2 Data cleaning and version selection

- **Population.** The major/minor releases from 10.0 on. Before 10.0 Guava used the r01...r09 numbering, and 10.0 is its first release with the current scheme.
- **Excluded releases.** Patch releases (x.y.z, z > 0), release candidates (-rc) and two special tags (13.0-final, 15.0-cdi1.0). The one exception is the last version: the assignment asks for the latest release, which is the patch release 33.7.2. Its bytecode has the same 1,948 class files and **exactly the same** DependencyExtractor output as the feature release 33.7.0, so the choice does not affect any structural result.
- **Sparse sample.** Every second major release (10, 12, ..., 32) and 33.7.2: **13 versions**. The time gaps vary from 6 months to 3.3 years (Table 1) because Guava's release rate slowed down after 2020; the version axis of all figures is ordinal, not time-proportional.
- **Classes.** module-info/package-info are excluded, and inner and anonymous classes are folded into their top-level class. The class inventories differ by step (Table 1; 33.7.2 as example):
 - **594** top-level classes in the jar;
 - **554** in the dependency data (the other 40 have no dependency on another Guava class and are not mentioned by any);
 - **549** connected, i.e. without 5 classes (Charsets, Flushables, Runnables, ...) whose only recorded dependencies are to themselves or to their own inner classes.
- **Populations of the architectures.** A1, A3, PKG and the AI architecture cluster the *connected* classes. A2, A4 and "PKG without noise" cluster the connected classes **minus the noise classes** (457 in 33.7.2). This includes the 52 classes whose only dependencies were to noise classes and which become isolated when the noise is removed. ACDC only sees classes that occur in a dependency, so each of them becomes a singleton cluster of A2.

2.3 Tools

Step	Tool / library
Class dependencies (3.2)	Course DependencyExtractor.jar
Widely-used classes (3.3)	Course JNode.jar (Constantinou et al. 2015), with documented fixes (tools/JNode-fixed.jar, see below)
ACDC (A1, A2)	ACDC (Tzerpos & Holt), Java port from USC ARCADE (tools/acdc.jar); the York download link was not reachable
k-means (A3, A4)	scikit-learn KMeans (as suggested in the tutorial)
MoJoFM	Original MoJo 2.0 implementation (tools/mojo.jar)
Metrics, plots	Python 3.13 (numpy, scipy, networkx, matplotlib)

JNode: defects found and fixed. JNode needed more than two hours for one Guava version. While speeding it up we decompiled it and found two defects in the construction of its class dependency graph. Details, sources and tests are in tools/jnode-patch/.

1. *Wrong dependencies (graph fix 1).* The dependency collector keeps one **static** map for all classes, and after reading class X JNode attached **every entry of that map** to X as out-dependencies. So each class inherited the dependencies of all classes read before it, and the graph depended on the order of the classes in the jar. Our test with four classes (A uses B, C uses D) in two archive orders gives $C \rightarrow \{A, B, C, D\}$ in one order and $B \rightarrow \{C, D\}$ in the other. For Guava 10.0 the original graph has **710,269 edges**, while DependencyExtractor finds **4,239**. The fix uses only X's own entry.
2. *Self-dependencies (graph fix 2).* A class name was compared with the target before converting a/b/c to a.b.c, so every class that refers to itself depended on itself (1,062 self-edges in 10.0).
3. *Precision.* JNode starts every class weight (step 1 of the method) at $\text{round}(1/n, 3)$. For **more than 2,000 class files** this rounds to **0**, every SIG becomes 0, and no class is flagged. This happens for Guava **30.0 (2,010 class files)**. We round to 4 decimals. On the other 12 versions, 3 and 4 decimals give exactly the same noise classes (Table 3).
4. *Speed.* Two linear scans were replaced by hash lookups, without changing the output.

With fixes 1 and 2, the graph inside JNode is **identical** to the DependencyExtractor graph (4,239 of 4,239 edges for 10.0 and 7,764 of 7,764 for 33.7.2, at class-file level). The original tool flagged **no class at all** for 10.0 and 12.0. The fixed tool flags 90 and 98 class files there, and **no threshold fallback is needed in any version**. Because the defects are in the course tool, we will report them to the instructor.

JNode threshold. We use JNode's own decision: a class is noise if its normalised SIG $\geq \text{mean} + 1\sigma$ (Lecture 6-7, step 4). A top-level class is noise if one of its class files is.

ACDC naming. ACDC names a subsystem after the base name of its dominator (text before the last dot), which for Java class names is the package, so all subsystems of a package would be merged in the output. We give ACDC file-style names (pkg.Class.java) and remove the suffix from the output.

2.4 Architecture recovery (A1-A4)

scripts/recover.py builds the dependency graph of the top-level classes of each version and writes the ACDC input (guava/data/rsf/<v>_full.rs and _nonoise.rs, one depends A B line per dependency). It then recovers:

- **A1 / A2** - ACDC on the full graph / on the graph without the classes flagged by JNode;
- **A3 / A4** - k-means on the full graph / on the graph without them;
- **PKG** - the developers' package structure, used as reference.

Features for k-means. k-means needs a numeric vector per class, but the input is a graph. The vector of a class is its row of the symmetric adjacency matrix $A + A^T + I$: which classes it uses, which classes use it, and itself. This is the dependency information that the other methods use. The direction is dropped because classes that collaborate belong together whichever way the dependency points, and the identity keeps classes without dependencies distinct. Rows are scaled to length 1, so that large hubs do not dominate. They are then reduced to 64 dimensions with truncated SVD, because k-means' Euclidean distance works poorly on sparse 500-dimensional vectors. After a second length normalisation, the distance between two vectors measures how different their dependency profiles are.

Number of clusters. For every version and both graphs we ran k-means for $k = 4 \dots n/3$ and chose k with the **elbow method**: the point where the within-cluster sum of squares (inertia) stops falling steeply (Figure 2), located automatically with the Kneedle algorithm. If no elbow is found, the script takes the k farthest from the straight line between the first and the last point of the curve; this fallback was not needed for any version. k ranges from 52 to 74 for A3 and from 34 to 68 for A4.

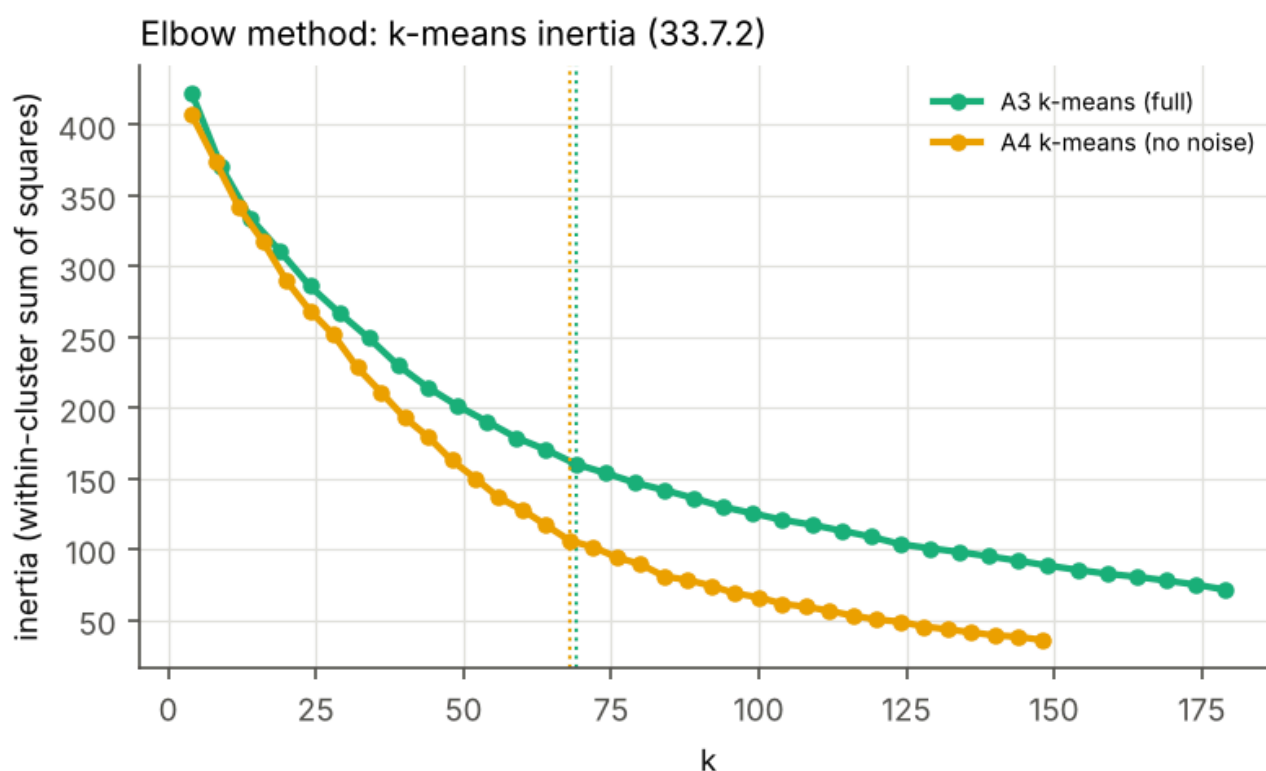


Figure 2 - Elbow method for 33.7.2; dotted lines = chosen k .

2.5 Evaluation

All measures are those of Lectures 1-2, 5 and 6-7.

- **Cohesion** $A_i = \mu_i / N_i^2$ (μ_i dependencies inside cluster i , N_i classes). The lecture allows μ_i / N_i^2 or $\mu_i / (N_i(N_i - 1))$. We use N_i^2 because it is defined for one-class clusters, where it is 0.
- **Coupling** $E_{ij} = \varepsilon_{ij} / (2 \cdot N_i \cdot N_j)$ (ε_{ij} dependencies between clusters i and j).

- **MQ (Modularization Quality)** = $(1/k) \cdot \sum A_i - (2/(k(k-1))) \cdot \sum E_{ij}$, in $[-1, 1]$.
- **MoJoFM**(A, R) = $(1 - \text{mno}(A, R) / \max \text{mno}(\cdot, R)) \cdot 100 \%$, where mno is the minimum number of Move and Join operations that turn architecture A into reference R. MoJo needs both partitions to contain the same classes, so every comparison is made on the **classes common to both** (e.g. classes present in both versions). Guava has no expert architecture, so the references are the **package structure** (every version) and the **AI architecture** of Phase 4 (33.7.2). MoJoFM between the architectures of consecutive versions shows how much the architecture changes (**stability**).
- **Widely-used classes**: JNode and, for comparison, the **Bunch** rule (in-degree > 3 × average in-degree), each followed by ACDC as in the lecture's comparison of "System", "Noise" and "Bunch".
- **Architectural smells** (Lecture 5), per version and normalised by the number of classes or packages:
 - *cyclic dependency*: packages/classes in a dependency cycle, found with depth-first search;
 - *hub-like dependency*: fan-in and fan-out above the medians and $|\text{in} - \text{out}| < 0.25 \cdot (\text{in} + \text{out})$;
 - *unstable dependency*: a package that depends on a less stable package, with instability $I = C_e / (C_a + C_e)$;
 - *god component*: a package with more classes than mean + 2σ of the package sizes (the lecture leaves the threshold open).
- **Lehman's laws** (Lecture 1-2): size, dependencies, dependencies added and removed per version (as in the lecture's Eclipse example), and commits per month between the analysed releases.

3. Results and discussion

3.1 Size

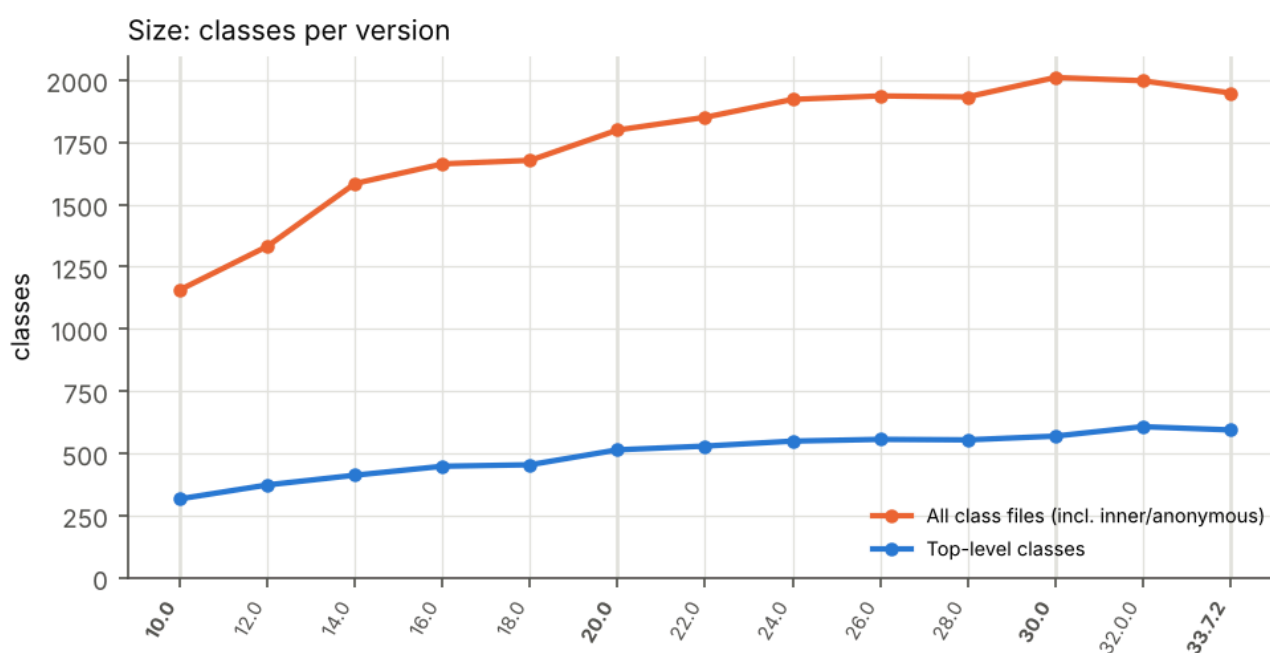


Figure 3 – Classes per version (versions on an ordinal axis).

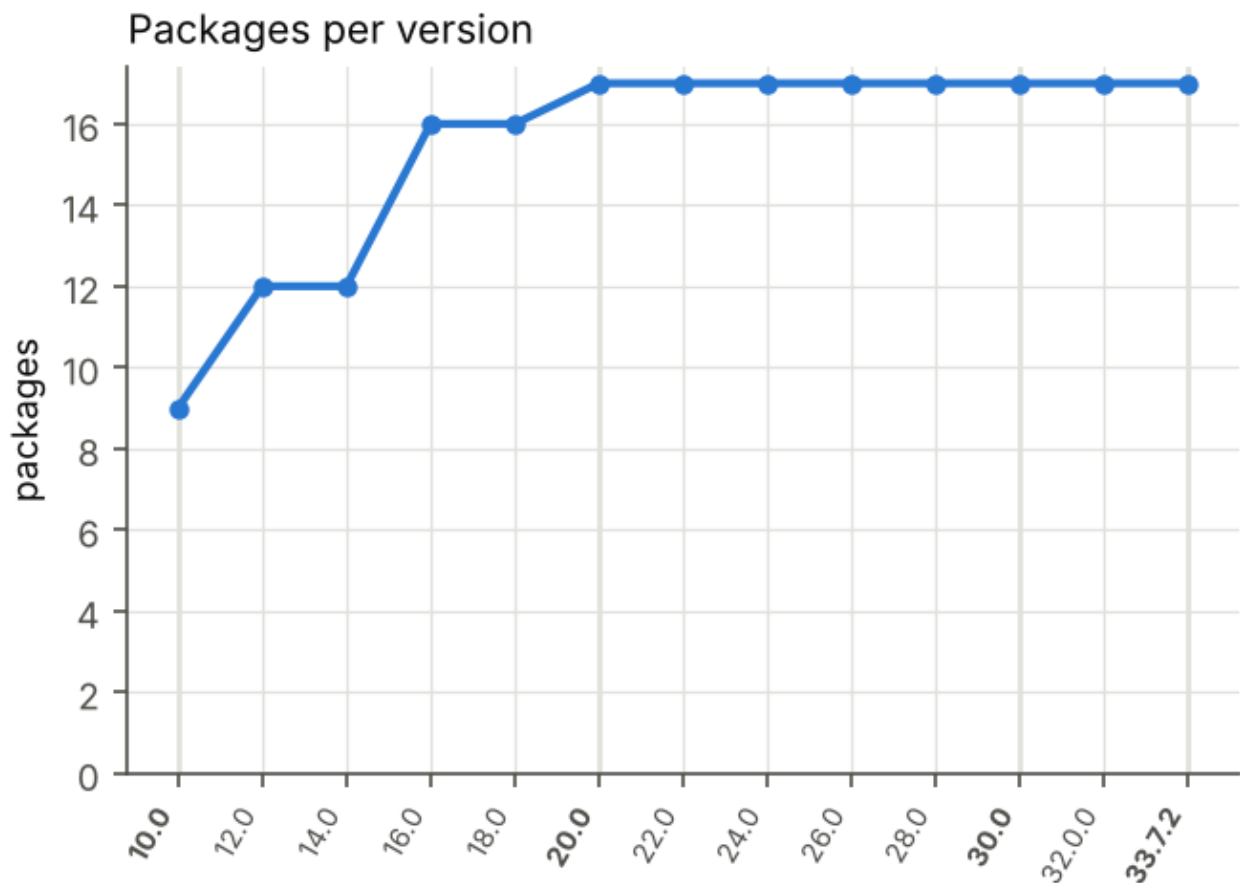


Figure 4 - Packages per version.

Guava grew from **317 to 594 top-level classes (+87 %)** and from 1,157 to 1,948 class files over 15 years. Growth is concentrated in the first five years:

- **10.0 → 20.0 (2011-2016): +62 %.** New packages were added:
 - hash, math and reflect in 12.0;
 - escape, html, xml and the public-suffix list in 16.0;
 - the graph library in 20.0 (+76 classes).
 The number of packages grew from 10 to 18.
- **20.0 → 32.0 (2016-2023): +18 %**, without new packages. The changes are additions to existing packages:
 - mainly Java 8 support between 20.0 and 24.0 (Streams, MoreCollectors, CollectSpliterators, FluentFuture, MoreFiles, ImmutableListArray);
 - graph traversal (Traverser).
- **32.0 → 33.7 (2023-2026): -2 %.** 14 classes were added and 18 removed (Table 9). The library is now in maintenance mode.

Lehman's laws.

- *Continuing change (I)* holds: every sampled release changes classes (Table 9).
- *Continuing growth (VI)* holds, but the rate falls strongly over time.
- Since 20.0 no sampled release added more than 7 % new classes. This is consistent with *conservation of familiarity (V)*: the size of the changes between releases stays small and bounded. Guava's compatibility policy reinforces this: public API can only be removed after deprecation, and new API must fit an existing package.

3.2 Class dependencies

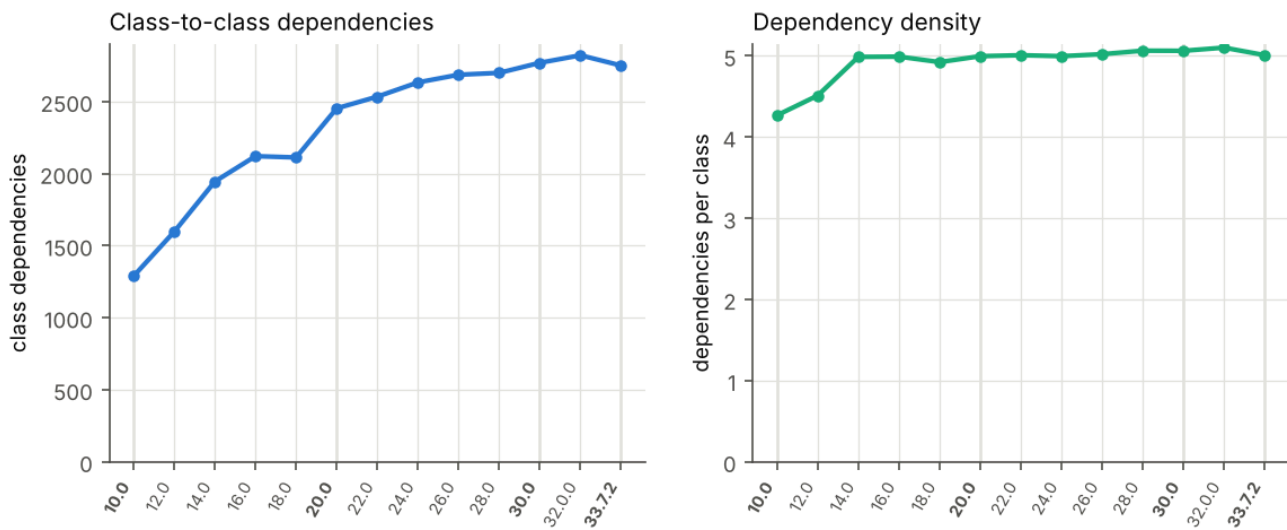


Figure 5 – Dependencies (left) and dependencies per connected class (right).

Dependencies grew from **1,289 to 2,750 ($\times 2.1$)**, faster than the connected classes ($\times 1.8$). The density rose from **4.27 to 5.0 dependencies per connected class by 20.0**. Since then it has stayed between 4.99 and 5.10 (peak in 32.0.0). It falls slightly in two steps: 16.0 $\rightarrow$ 18.0 (4.99 $\rightarrow$ 4.92) and 32.0.0 $\rightarrow$ 33.7.2 (5.10 $\rightarrow$ 5.01). So complexity increases strongly during the expansion phase and then levels off, without being reduced. This is consistent with *increasing complexity (II)*: new features are built on top of the existing collection and base APIs, and there is no release that lowers the density noticeably.

3.3 Widely-used (noise) classes

Table 3 – Widely-used classes (JNode) per version; comparison of JNode with the original 3-decimal and the 4-decimal CR model (class files).

Version	Connected classes	Isolated	Noise (top-level)	Noise %	Connected – noise	Isolated after removal	JNode-3 noise files	JNode-4 noise files	Jaccard 3 vs 4
10.0	302	8	61	20.2%	241	37	90	90	1.0
12.0	354	10	71	20.06%	283	45	98	98	1.0
14.0	390	9	74	18.97%	316	50	110	110	1.0
16.0	425	10	79	18.59%	346	58	117	117	1.0
18.0	429	10	82	19.11%	347	53	124	124	1.0
20.0	491	8	90	18.33%	401	52	141	141	1.0
22.0	506	8	94	18.58%	412	56	142	142	1.0
24.0	527	7	94	17.84%	433	52	149	149	1.0
26.0	535	6	85	15.89%	450	53	121	121	1.0
28.0	533	6	84	15.76%	449	52	119	119	1.0
30.0	547	6	94	17.18%	453	54	0	138	0.0
32.0.0	553	5	95	17.18%	458	55	132	132	1.0
33.7.2	549	5	92	16.76%	457	52	123	123	1.0

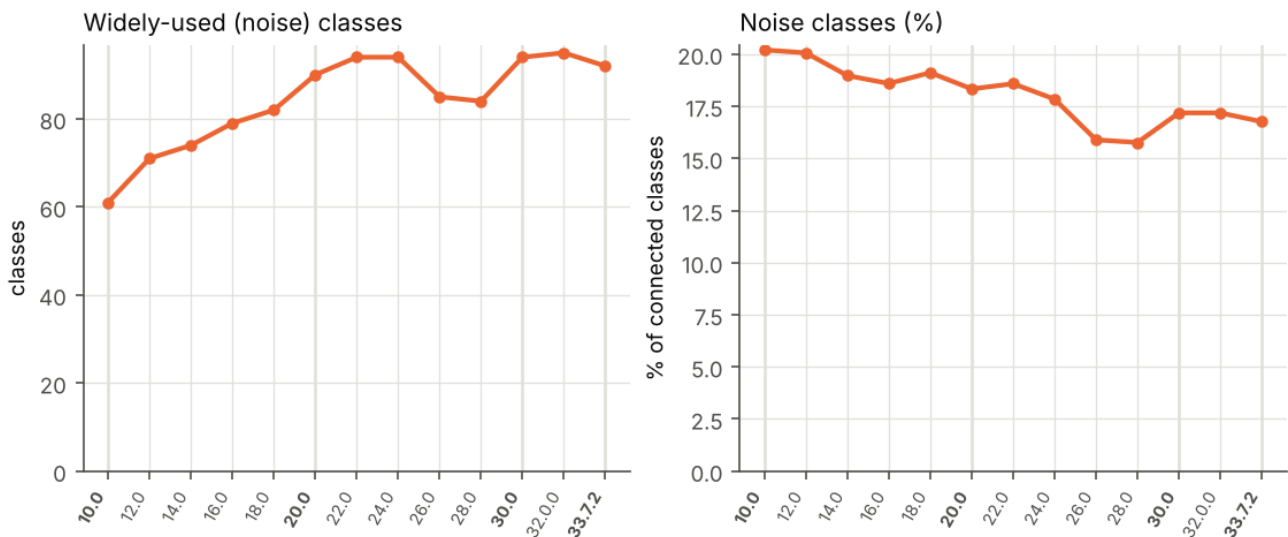


Figure 6 – Widely-used (noise) classes per version.

JNode flags **61-95 top-level classes, 16-20 % of the connected classes**. They are exactly what the lecture calls omnipresent classes:

- **Where they are.** In 33.7.2, 70 of the 92 are in `common.collect` and 11 in `common.base`. The rest are in `math` (4), `util.concurrent` (3), `primitives` (2), `graph` and `hash` (1 each).
- **Most significant classes:** `Preconditions`, `ImmutableList`, `Maps`, `Function`, `UnmodifiableIterator`, `ImmutableMap`, `ImmutableSet`, `Iterators`, `Multiset`, `Predicate`, `Supplier`, `Ordering`, `Sets`, `Multimap`, ...
- **How used they are.** A noise class is used by 19.1 other classes on average, other classes by 2.2. The noise classes take part in **68-76 % of all dependencies** (1,886 of 2,750 in 33.7.2).

Removing them therefore removes most of the graph. Out of 549 classes, 457 remain with only 864 dependencies, and 52 of the remaining classes lose all their dependencies (Table 3). This is the main effect behind A2 and A4 in the following sections.

3.4 Recovered architectures (A1-A4)

Table 4 – Mean over the 13 versions. The MoJoFM of A2/A4 is computed against the packages of the same population (without noise).

Architecture	k	Largest cluster	Cohesion	Coupling	MQ	MoJoFM to packages (%)
A1 ACDC (full)	78	46	0.2578	0.00649	0.2513	70.9
A2 ACDC (no noise)	119	28	0.1331	0.00069	0.1324	60.3
A3 k-means (full)	60	31	0.1756	0.00662	0.1690	73.9
A4 k-means (no noise)	56	17	0.1744	0.00116	0.1733	73.7
Packages (full)	15	205	0.1053	0.00503	0.1003	-
Packages (no noise)	15	140	0.0994	0.00285	0.0965	-

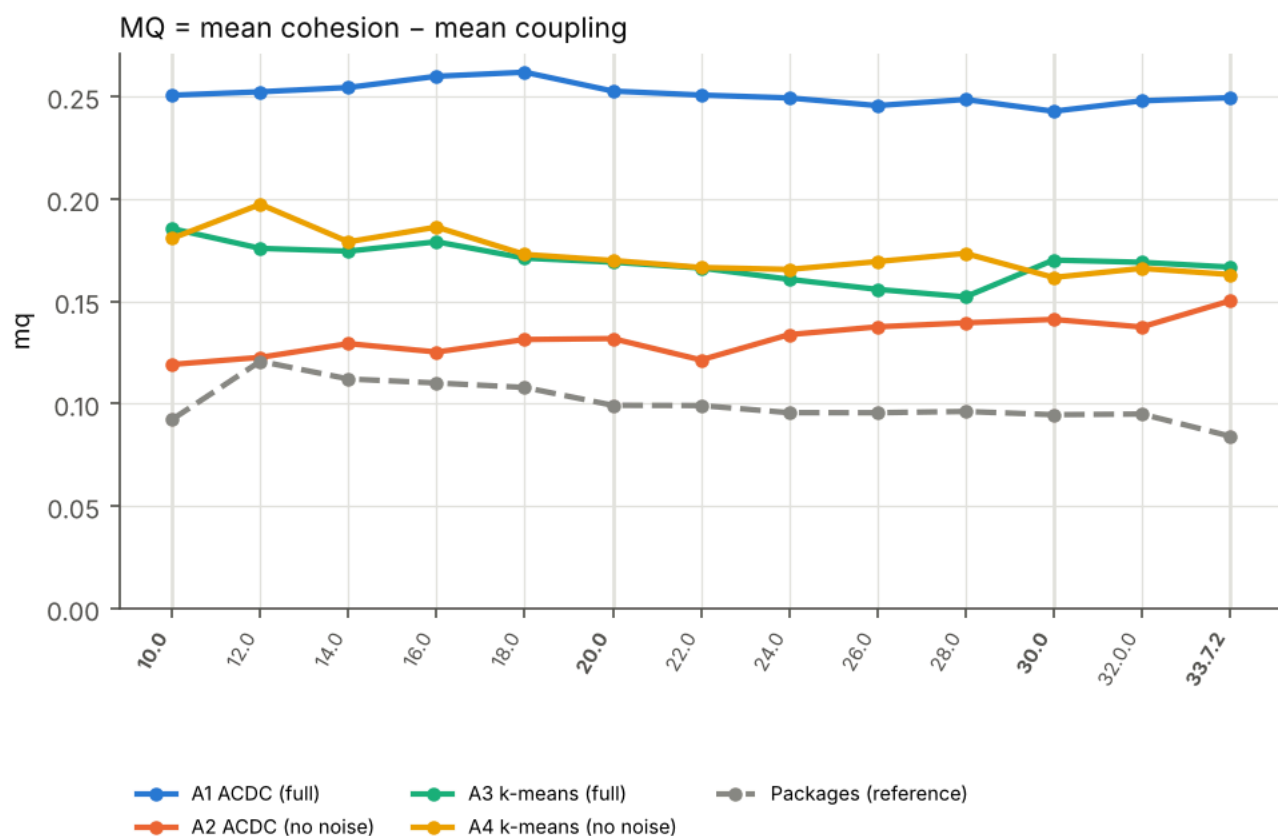


Figure 7 - MQ per version and architecture.

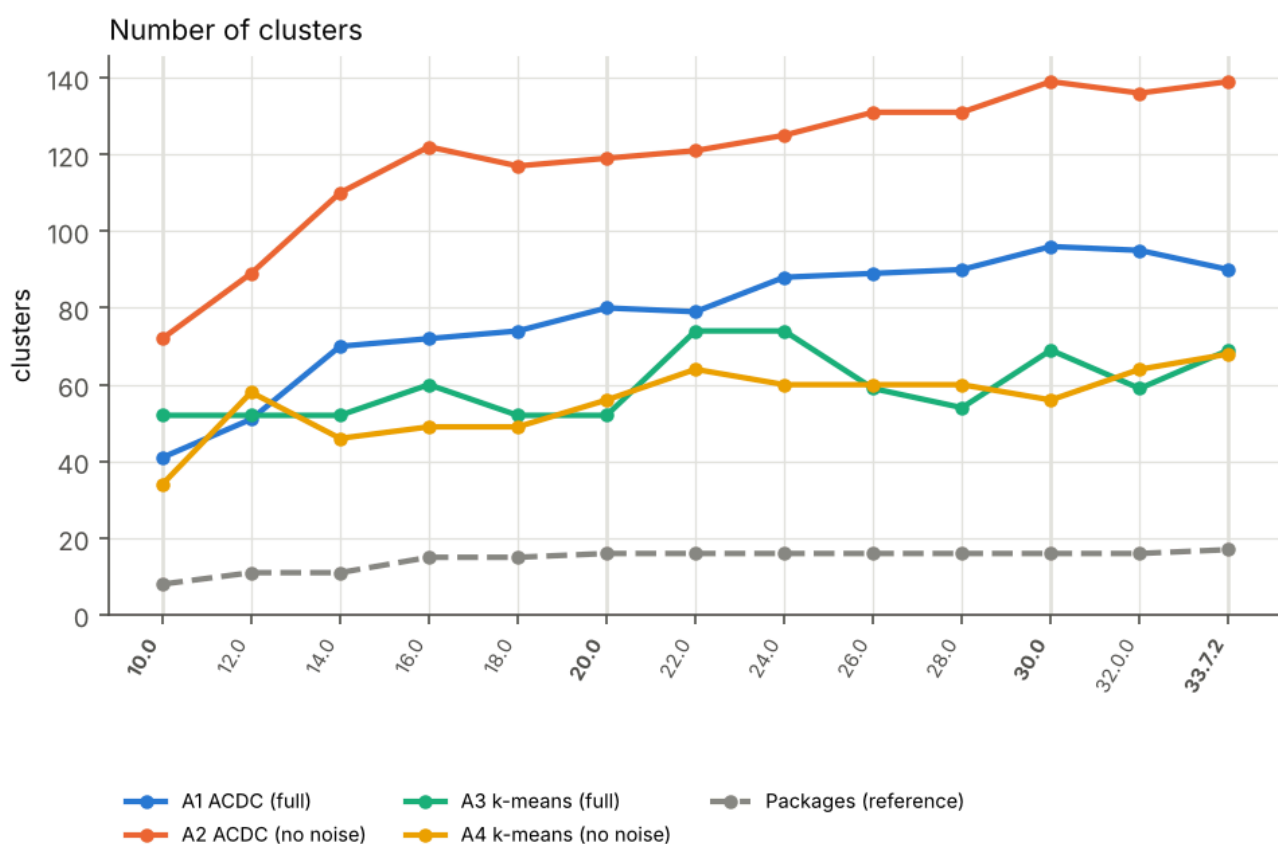


Figure 8 - Number of clusters.

Size.

- ACDC produces 41 (10.0) to 90–96 clusters (30.0–33.7.2), with a largest cluster of 27–52 classes.
- k-means produces 52–74 clusters of at most 43 classes.
- Without noise, ACDC produces more and smaller clusters: 72–139 clusters of 3 classes on average, 37–57 of them singletons (the isolated classes of 2.2).
- The packages are very coarse: 8–17 groups, the largest being `common.collect` with 179–217 classes.

Quality (MQ). On the full graph, ACDC has the highest MQ in every version (mean 0.251), k-means follows (0.169), and the package structure is last (0.100). Coupling is small for all of them (at most 0.0104), so MQ is decided by cohesion:

- ACDC’s clusters of 6–7 classes are dense (cohesion 0.258);
- k-means’ slightly larger ones are less so (0.176);
- the packages, with `common.collect` (~200 classes) as one group, have the lowest cohesion (0.105).

By the course’s measure, the **recovered architectures are better modularised than the package structure**. ACDC finds dense groups of collaborating classes inside the large packages (e.g. `Multimaps.ss`, the `multimap` implementations and their utilities; `Hashing.ss`, the hash functions).

- **ACDC vs k-means.** ACDC has the higher cohesion and MQ in every version, and the lower coupling in 7 of the 13 versions (33.7.2: 0.0048 vs 0.0096).
- **Removing the widely-used classes (A1→A2, A3→A4) removes almost all coupling:** ACDC 0.0065 → 0.0007, k-means 0.0066 → 0.0012. Classes such as `Preconditions` and `ImmutableList`, used by almost every cluster, no longer connect the clusters. MQ reacts differently for the two algorithms:
 - For **k-means** MQ stays the same (0.169 → 0.173).
 - For **ACDC** MQ falls (0.251 → 0.132). This is not a worse decomposition of the remaining graph. About 50 classes left without any dependency become singleton clusters with cohesion 0, which halves the mean cohesion.

MQ, as an unweighted average over clusters, is very sensitive to such clusters.

ACDC parameters (tutorial). Table 6 and Figure 9 show the experiment on 33.7.2:

- **BodyHeader has no effect** (`bso = so`).
- **The maximum cluster size of SubGraph has practically no influence:** 5 gives 91 clusters and 10 or more give 90. Guava’s dominator subgraphs are naturally small.
- **Without OrphanAdoption**, `ClusterLast` puts all orphans into **one cluster of 276 classes** (half of the system). MQ even *rises* (0.249 → 0.292), because MQ is an average over clusters: one huge, sparse cluster lowers it only once, while the remaining clusters stay small and dense. This shows a limitation of MQ. It has to be read together with the cluster sizes, and a catch-all cluster is not an architecture. We therefore keep the default (20, `bso`). The graph without noise behaves the same way (catch-all cluster of 166 classes).

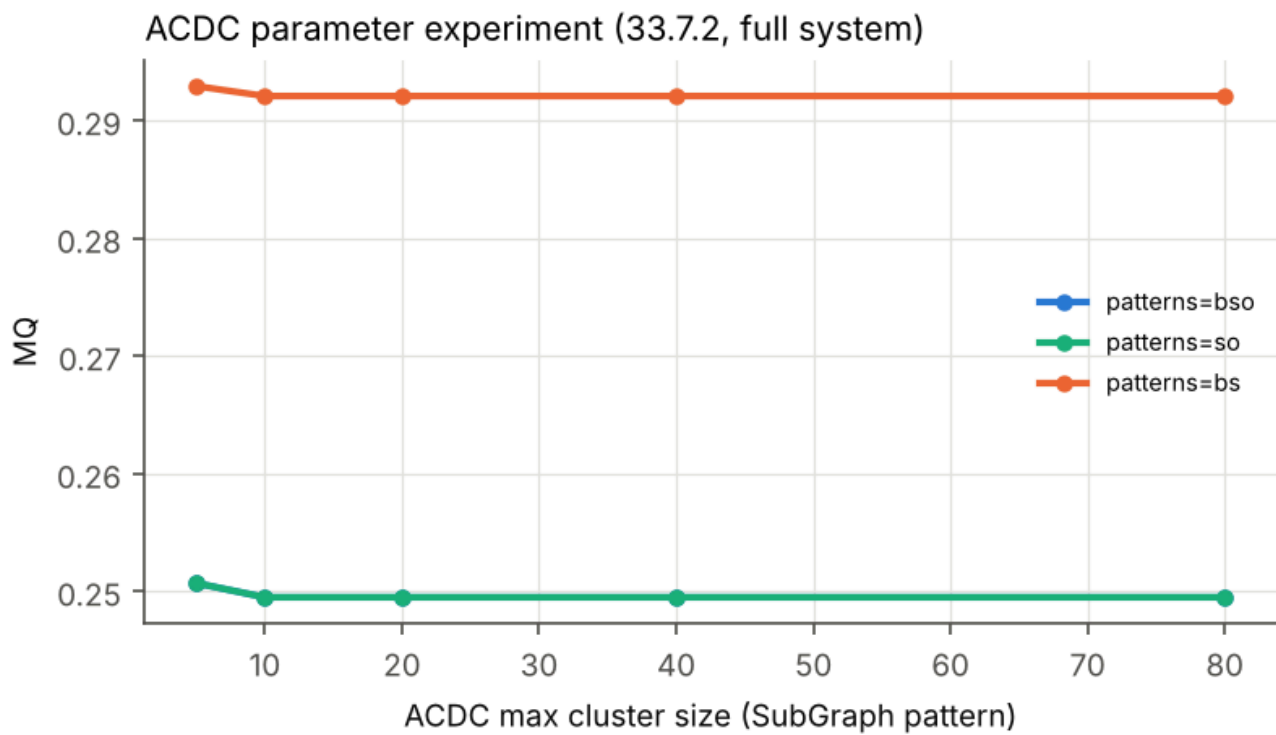


Figure 9 – ACDC parameter experiment (33.7.2, full graph); bso lies under so.

Table 6 – ACDC parameter experiment (33.7.2).

Graph	Patterns	Max size	k	Largest	Cohesion	Coupling	MQ
full	bso	5	91	47	0.25538	0.004713	0.25067
full	bso	10	90	47	0.25423	0.004754	0.24947
full	bso	20	90	47	0.25423	0.004754	0.24947
full	bso	40	90	47	0.25423	0.004754	0.24947
full	bso	80	90	47	0.25423	0.004754	0.24947
full	so	5	91	47	0.25538	0.004713	0.25067
full	so	10	90	47	0.25423	0.004754	0.24947
full	so	20	90	47	0.25423	0.004754	0.24947
full	so	40	90	47	0.25423	0.004754	0.24947
full	so	80	90	47	0.25423	0.004754	0.24947
full	bs	5	92	276	0.2986	0.005687	0.29291
full	bs	10	91	276	0.2978	0.005728	0.29207
full	bs	20	91	276	0.2978	0.005728	0.29207
full	bs	40	91	276	0.2978	0.005728	0.29207
full	bs	80	91	276	0.2978	0.005728	0.29207
nonoise	bso	5	140	25	0.15259	0.000846	0.15175
nonoise	bso	10	139	25	0.15111	0.000844	0.15026
nonoise	bso	20	139	25	0.15111	0.000844	0.15026
nonoise	bso	40	139	25	0.15111	0.000844	0.15026
nonoise	bso	80	139	25	0.15111	0.000844	0.15026
nonoise	so	5	140	25	0.15259	0.000846	0.15175
nonoise	so	10	139	25	0.15111	0.000844	0.15026
nonoise	so	20	139	25	0.15111	0.000844	0.15026
nonoise	so	40	139	25	0.15111	0.000844	0.15026
nonoise	so	80	139	25	0.15111	0.000844	0.15026
nonoise	bs	5	140	166	0.17375	0.000925	0.17282
nonoise	bs	10	139	166	0.17233	0.000918	0.17141
nonoise	bs	20	139	166	0.17233	0.000918	0.17141
nonoise	bs	40	139	166	0.17233	0.000918	0.17141
nonoise	bs	80	139	166	0.17233	0.000918	0.17141

3.5 Quality of the architecture over time

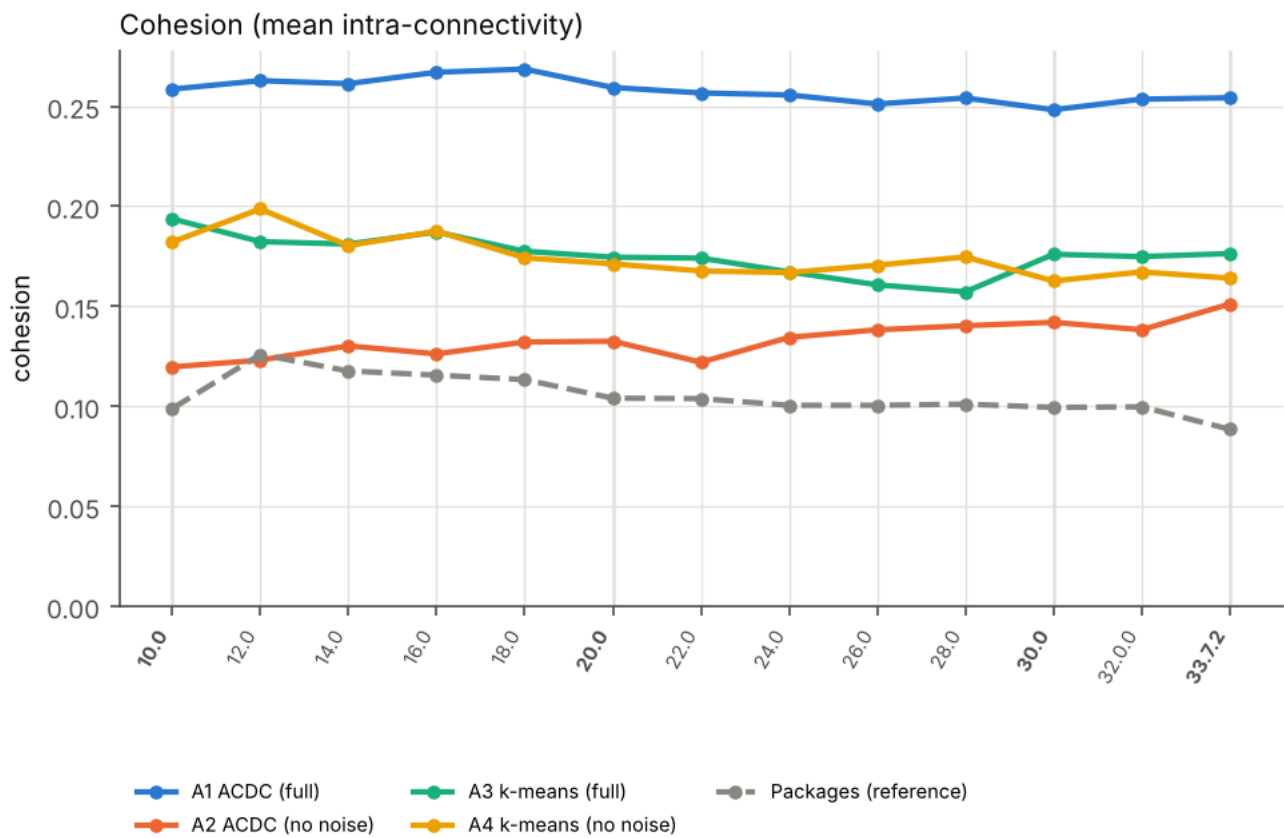


Figure 10 - Cohesion.

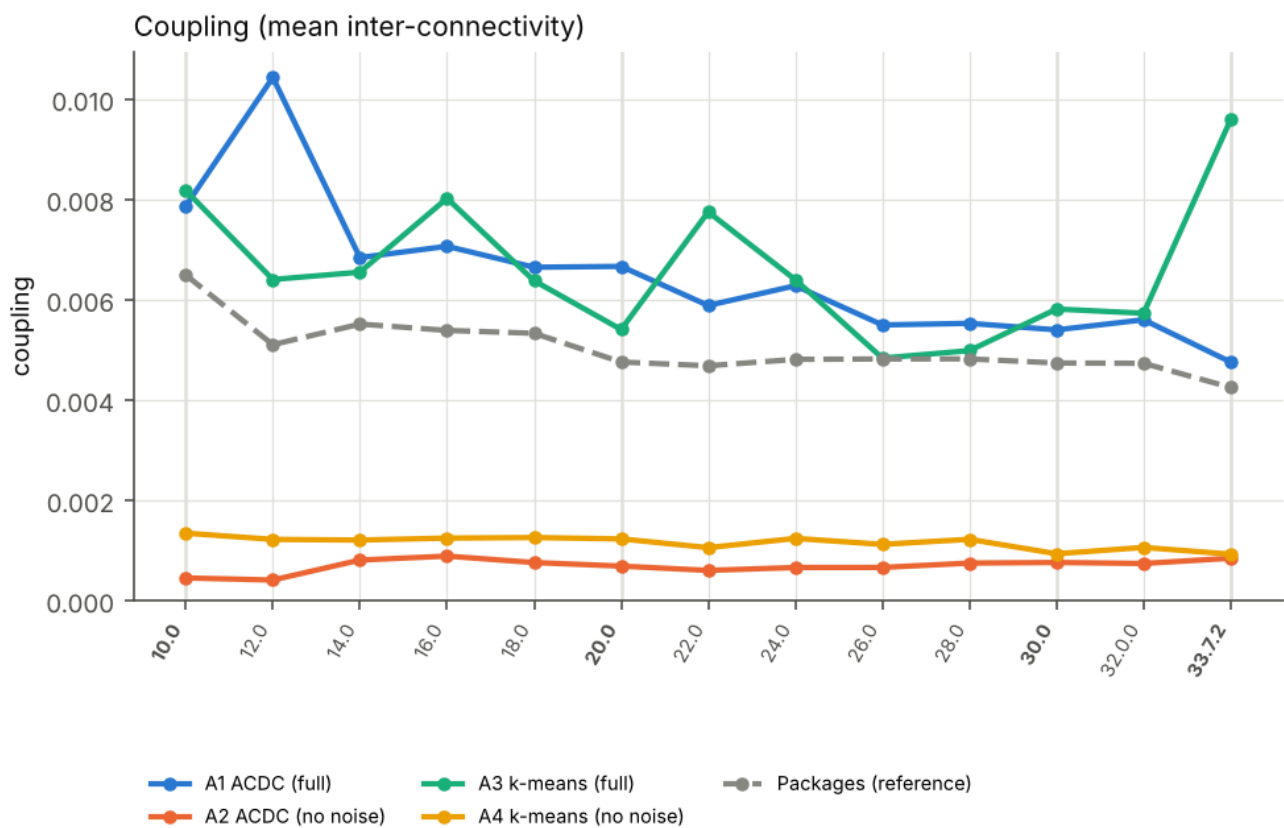


Figure 11 - Coupling.

1. **10.0 → 18.0 - expansion.** ACDC's MQ rises slightly (0.251 → 0.262): the new packages (hash, math, reflect, escape, ...) form well-separated, dense groups. The package structure reaches its best MQ in 12.0 (0.121), when hash, math and reflect were added as small, cohesive packages.
2. **18.0 → 30.0 - decline, then a small recovery.**
 - *ACDC*. MQ falls from 0.262 to its minimum of 0.243 in 30.0, then rises again slightly (0.248 in 32.0.0, 0.249 in 33.7.2).
 - *Packages*. MQ falls from 0.108 to 0.095. It is almost flat between 24.0 and 32.0.0, with small increases in 28.0 and 32.0.0, and reaches its lowest value, **0.084**, in 33.7.2. Their cohesion falls from 0.126 (12.0) to 0.088, because the large packages (collect, util.concurrent, graph) grow without adding proportionally many internal dependencies.
 - *k-means*. MQ fluctuates without a clear trend (A3 0.152–0.186, A4 0.162–0.197).
 - *ACDC without noise (A2)*. MQ rises over time (0.119 → 0.150), partly because the share of classes isolated by the noise removal falls (15 % → 11 %).
3. **Coupling** falls for ACDC (0.0079 → 0.0048) and the packages (0.0065 → 0.0042). Since E_{ij} is divided by $N_i N_j$, this mostly reflects the larger clusters and the larger number of cluster pairs, not a real decoupling.

The decline of MQ after the expansion phase, together with the rising dependency density (3.2), is **consistent with declining quality (VII)**. It does not prove it: the changes are small (ACDC 0.262 → 0.249), MQ depends on the clustering, and the latest releases even improve slightly for ACDC. Part of the explanation is Guava's compatibility policy. Packages and public classes are API, so a re-modularisation of the package structure would break clients. That makes such a restructuring costly and unlikely, but not impossible: internal classes can be, and are, reorganised (e.g. the removal of 18 classes in 33.7).

3.6 Comparison with the reference architecture (MoJoFM)

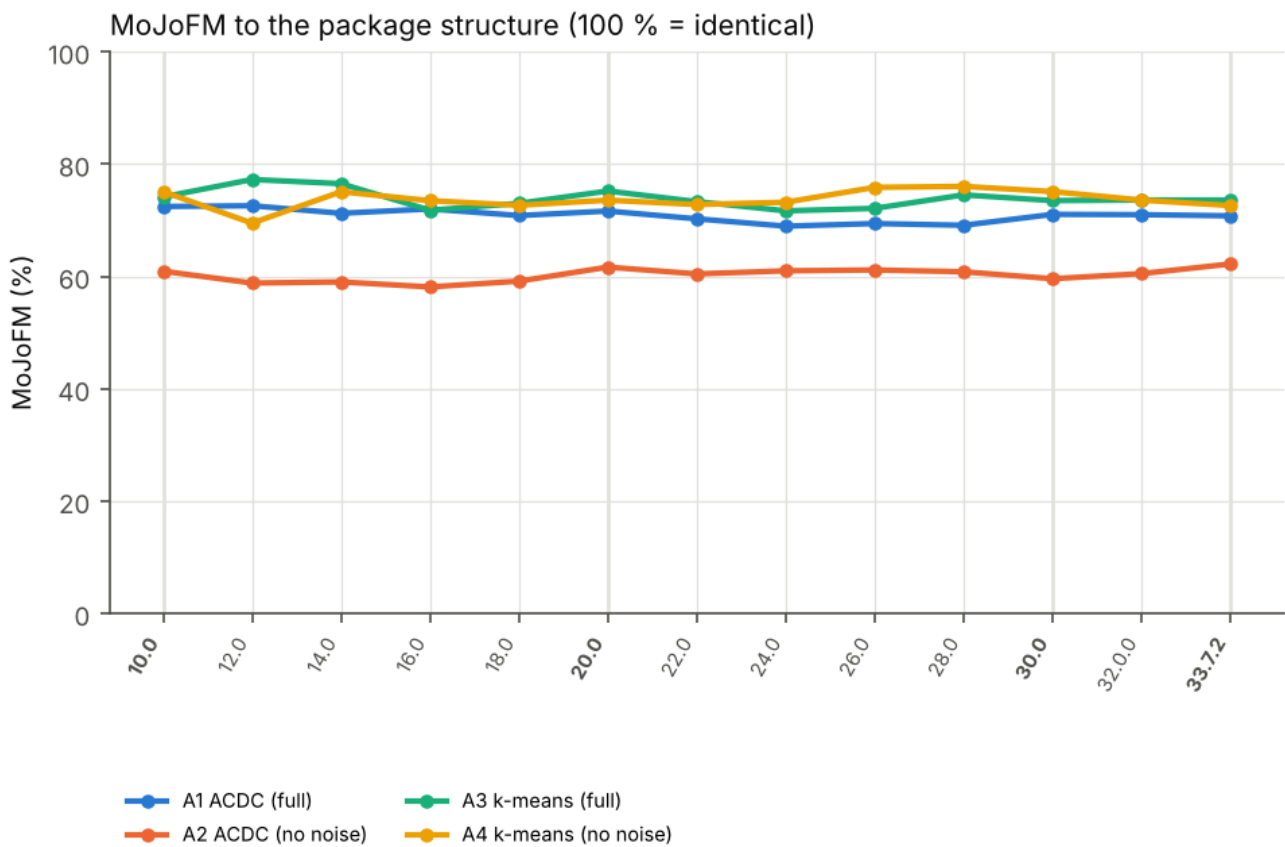


Figure 12 - MoJoFM of A1-A4 to the package structure.

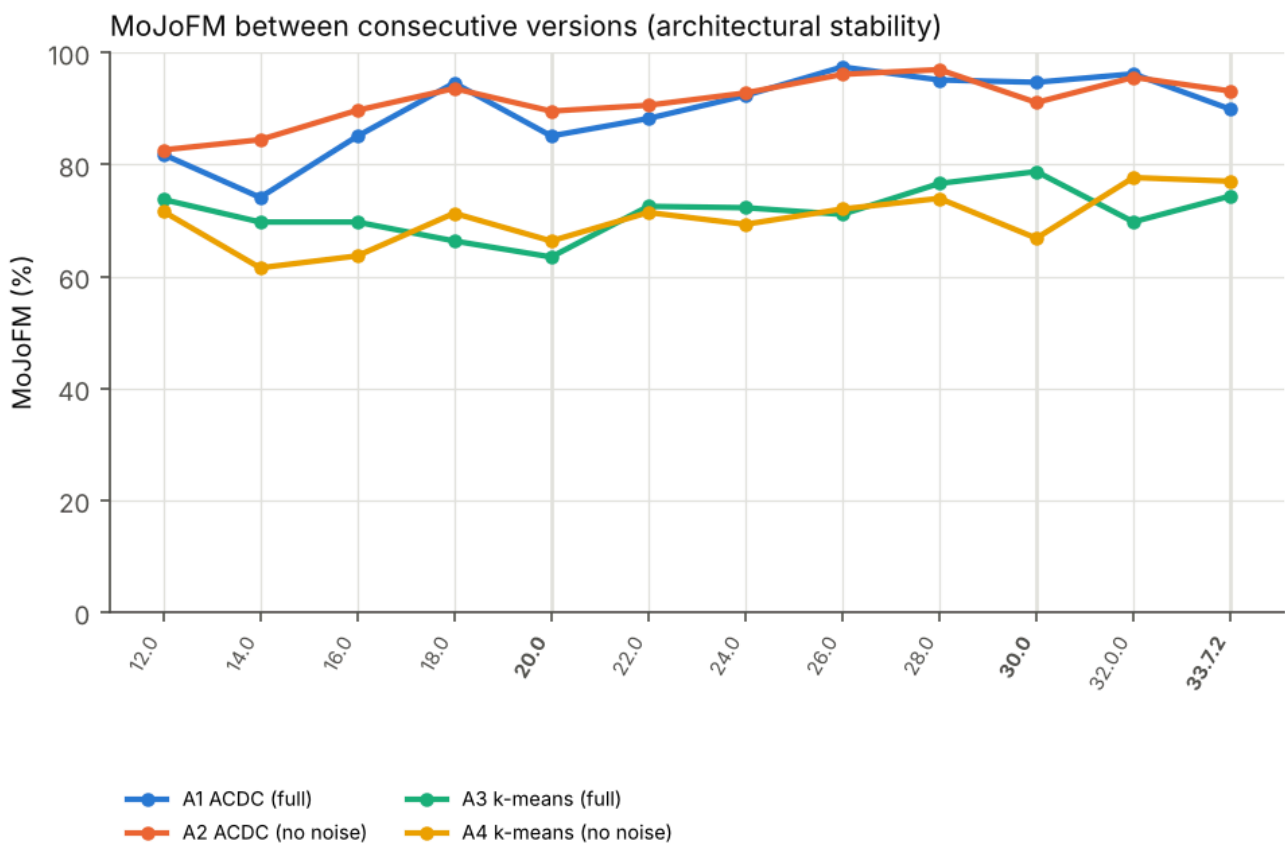


Figure 13 - MoJoFM between consecutive sampled versions (stability).

- **To the packages** (Table 13), on the full graph, the recovered architectures are 69–77 % similar to the packages. **k-means is closer than ACDC** (A3 73.9 % vs A1 70.9 % on average).
- **Removing the widely-used classes** does not bring the architectures closer to the packages: A4 stays at 73.7 % and A2 falls to 60.3 %. The fall of A2 is again caused by the singletons. Each class isolated by the noise removal costs one Join operation. On the classes that keep a dependency (“linked” columns), A2 is as close as A1 (70.1 %) and **A4 is closer than A3** (76.6 % vs 73.9 %). So only the k-means result shows the improvement that the lecture reports for the removal of omnipresent classes.
- **To the AI architecture** (33.7.2, Table 14), the packages are closest (70.5 %), then k-means (67.5 %, 67.3 % without noise), then ACDC (64.9 %, 60.7 %). Conversely, the AI architecture is 94.4 % similar to the packages. It differs mainly by splitting `collect` and `util.concurrent` and by grouping small packages.
- **Stability** (Table 15). ACDC is much more stable than k-means (mean MoJoFM 89.5 % / 91.3 % vs 71.5 % / 70.2 %).
 - ACDC’s stability grows as the library matures: 74–85 % in the expansion steps (12.0–16.0 and 20.0) and 92–97 % between 24.0 and 32.0.0.
 - The least stable step is 12.0 → 14.0, where 58 classes were added and 23 removed.
 - Values are lower than they would be for consecutive releases, because each step spans two majors (up to 3.3 years).

Table 13 – MoJoFM (%) of the recovered architectures to the package structure. A2/A4 are compared with the packages of the classes without noise. “Linked” = only the classes that keep at least one dependency after the noise removal.

Version	A1 → PKG	A2 → PKG	A3 → PKG	A4 → PKG	A2 → PKG (linked)	A4 → PKG (linked)
10.0	72.45	60.94	74.15	75.11	72.45	79.08
12.0	72.59	58.82	77.26	69.49	70.04	74.01
14.0	71.24	59.02	76.52	75.08	70.59	78.43
16.0	72.02	58.13	71.78	73.49	70.44	76.64
18.0	70.84	59.16	73.01	72.67	70.36	75.36
20.0	71.64	61.66	75.21	73.58	70.96	76.35
22.0	70.26	60.45	73.32	72.8	70.38	76.83
24.0	68.95	61.0	71.68	73.21	69.67	74.86
26.0	69.42	61.15	72.12	75.86	69.63	76.96
28.0	69.11	60.83	74.52	76.04	69.11	77.49
30.0	71.05	59.59	73.5	75.11	67.97	77.08
32.0.0	71.0	60.5	73.61	73.59	69.07	76.55
33.7.2	70.79	62.22	73.6	72.62	70.51	75.9

Table 14 – MoJoFM (%) to the AI architecture (33.7.2), on the classes common to both partitions.

MoJoFM (%) to the AI architecture	A1	A2	A3	A4	Packages
33.7.2	64.86	60.72	67.48	67.27	70.47
Classes compared	549	457	549	457	549

Table 15 – MoJoFM (%) between consecutive sampled versions (classes present in both versions).

Version (vs previous)	A1	A2	A3	A4
12.0	81.82	82.61	73.76	71.63
14.0	74.12	84.43	69.72	61.54
16.0	85.04	89.68	69.72	63.67
18.0	94.57	93.58	66.34	71.3
20.0	85.1	89.54	63.48	66.34
22.0	88.22	90.59	72.53	71.43
24.0	92.32	92.78	72.28	69.31
26.0	97.44	96.11	71.09	72.05
28.0	95.11	96.96	76.61	73.95
30.0	94.69	91.08	78.74	66.83
32.0.0	96.2	95.58	69.75	77.67
33.7.2	89.88	93.13	74.37	77.01

3.7 Widely-used classes: JNode vs Bunch

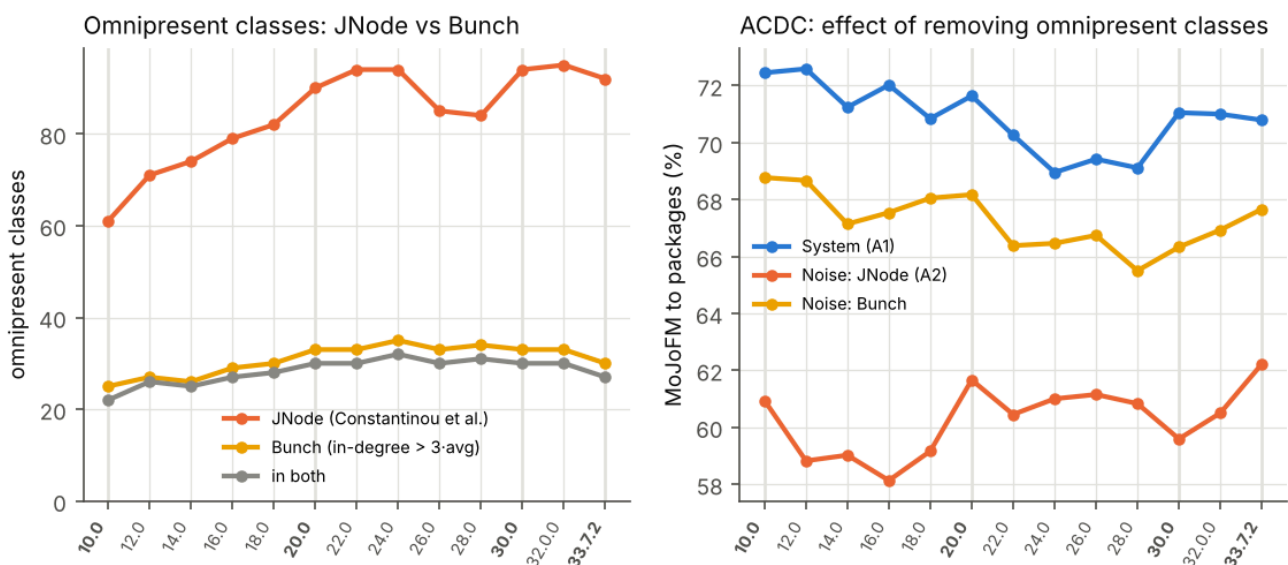


Figure 14 - Widely-used classes by JNode and by the Bunch rule (left); MoJoFM of ACDC to the packages for the full system, without JNode's and without Bunch's classes (right).

The Bunch rule (in-degree > 3 × average) flags **25-35 classes**. In 33.7.2 these are, for example:

- from base: Preconditions, Function, Predicate, Supplier, MoreObjects;
- from collect: ImmutableList/Set/Map, Iterators, Iterables, Lists, Maps, Sets, Ordering, Multimaps, Multiset;
- from other packages: hashCode, Ints, ListenableFuture, EndpointPair.

88-96 % of them are also flagged by JNode, so the Bunch set is almost a subset of JNode's. JNode's set is about three times larger, which keeps the Jaccard index at 0.28-0.36. JNode adds classes that are less used but lie on many dependency paths, mainly in collect. As the lecture notes, the Bunch rule depends on its threshold (3 × average). JNode needs no threshold but flags more classes.

For ACDC (Table 16):

- Removing Bunch's smaller set leaves fewer isolated classes than removing JNode's. ACDC on the rest therefore has a higher MQ (0.14–0.19 vs 0.12–0.15) and is closer to the packages (65.5–68.8 % vs 58.1–62.2 %).
- Neither variant brings ACDC closer to the packages than the full system (68.9–72.6 %).

Table 16 – JNode vs Bunch and their effect on ACDC.

Version	Bunch (>3·avg)	JNode	Both	Jaccard	MQ System	MQ JNode	MQ Bunch	MoJoFM System	MoJoFM JNode	MoJoFM Bunch
10.0	25	61	22	0.344	0.251	0.119	0.144	72.45	60.94	68.77
12.0	27	71	26	0.361	0.252	0.123	0.152	72.59	58.82	68.67
14.0	26	74	25	0.333	0.254	0.129	0.167	71.24	59.02	67.14
16.0	29	79	27	0.333	0.260	0.125	0.182	72.02	58.13	67.54
18.0	30	82	28	0.333	0.262	0.131	0.190	70.84	59.16	68.05
20.0	33	90	30	0.323	0.253	0.132	0.171	71.64	61.66	68.17
22.0	33	94	30	0.309	0.251	0.121	0.174	70.26	60.45	66.38
24.0	35	94	32	0.33	0.249	0.134	0.173	68.95	61.0	66.46
26.0	33	85	30	0.341	0.246	0.138	0.177	69.42	61.15	66.74
28.0	34	84	31	0.356	0.249	0.140	0.180	69.11	60.83	65.5
30.0	33	94	30	0.309	0.243	0.141	0.179	71.05	59.59	66.33
32.0.0	33	95	30	0.306	0.248	0.138	0.181	71.0	60.5	66.93
33.7.2	30	92	27	0.284	0.249	0.150	0.186	70.79	62.22	67.66

3.8 Architectural smells and their evolution

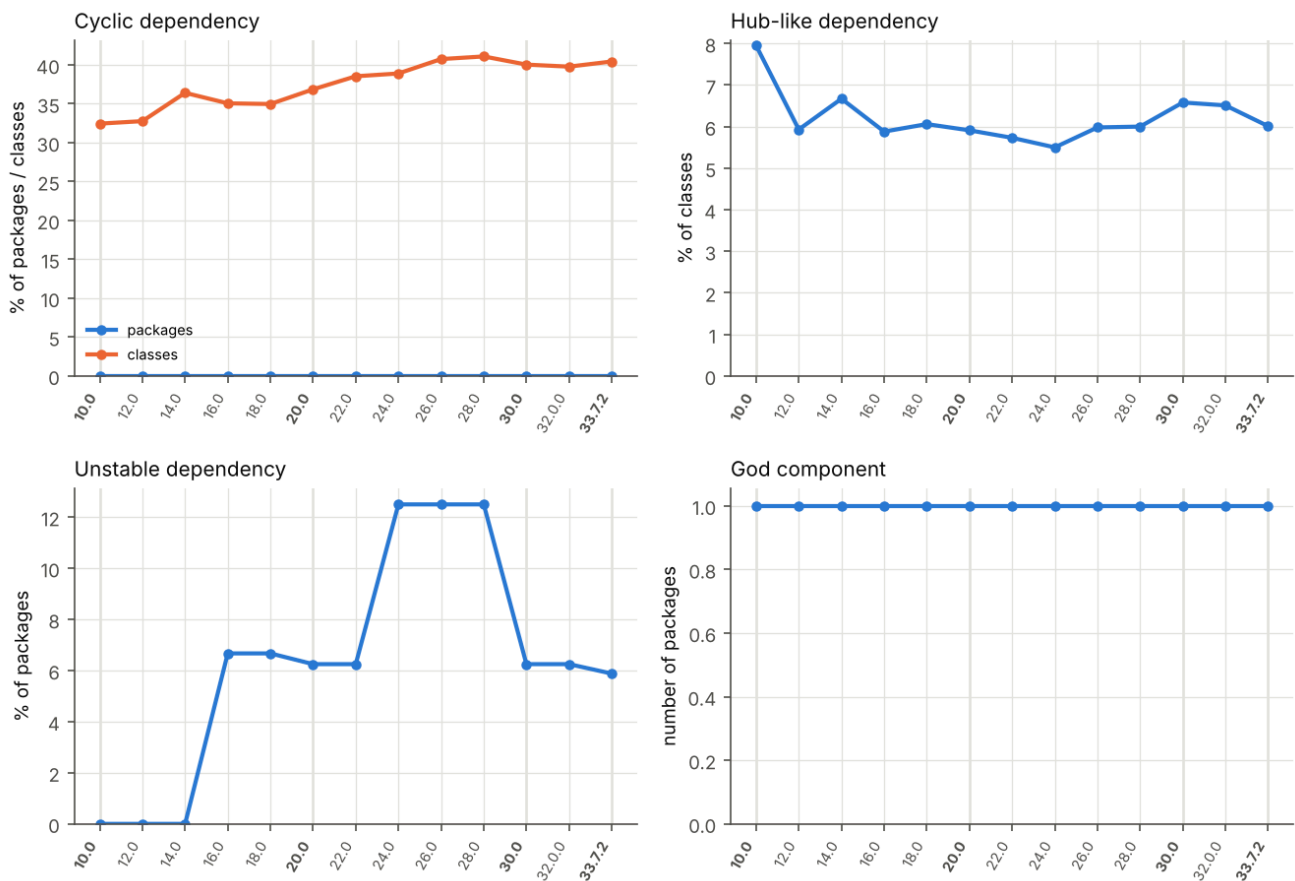


Figure 15 – Evolution of the architectural smells.

- **Cyclic dependency - none between packages, many inside them.** Guava's package dependency graph is **acyclic in all 13 versions**. This is a deliberate layering (base → primitives → collect → concurrency → features) kept for 15 years. Inside the packages, however, the share of classes in a dependency cycle rises from **32 % to 41 %** (28.0), then stays at 40 %. Almost all of them are in `collect`, where interfaces, implementations and utilities reference each other (`ImmutableList` ⇌ `ImmutableCollection` ⇌ `Iterators`, ...). The package boundaries stay clean while the entanglement grows inside them.
- **Hub-like dependency** affects 5.5–8 % of the classes, highest in 10.0 and then stable at ≈ 6 %.
- **Unstable dependency** appears in 16.0. `collect` depends on the less stable `math` (16.0–28.0), and from 24.0 `io` depends on the less stable `graph` (the file-tree traversal of `MoreFiles/Files` uses `graph.Traverser`). At most 2 packages (12.5 %) have the smell. In 33.7.2 only `io` remains.
- **God component:** `common.collect` in **every** version (threshold 130–146 classes; `collect` has 179–217). It is the main architectural smell of Guava, and it is hard to remove because its classes are public API.

Table 17 – Architectural smells per version.

Version	Cyclic pkgs %	Cyclic classes %	Hub-like %	Unstable dep. %	God comp.	God components
10.0	0.0	32.45	7.95	0.0	1	<code>common.collect</code>
12.0	0.0	32.77	5.93	0.0	1	<code>common.collect</code>
14.0	0.0	36.41	6.67	0.0	1	<code>common.collect</code>
16.0	0.0	35.06	5.88	6.67	1	<code>common.collect</code>
18.0	0.0	34.97	6.06	6.67	1	<code>common.collect</code>
20.0	0.0	36.86	5.91	6.25	1	<code>common.collect</code>
22.0	0.0	38.54	5.73	6.25	1	<code>common.collect</code>
24.0	0.0	38.9	5.5	12.5	1	<code>common.collect</code>
26.0	0.0	40.75	5.98	12.5	1	<code>common.collect</code>
28.0	0.0	41.09	6.0	12.5	1	<code>common.collect</code>
30.0	0.0	40.04	6.58	6.25	1	<code>common.collect</code>
32.0.0	0.0	39.78	6.51	6.25	1	<code>common.collect</code>
33.7.2	0.0	40.44	6.01	5.88	1	<code>common.collect</code>

3.9 Lehman's laws: dependency changes and work rate

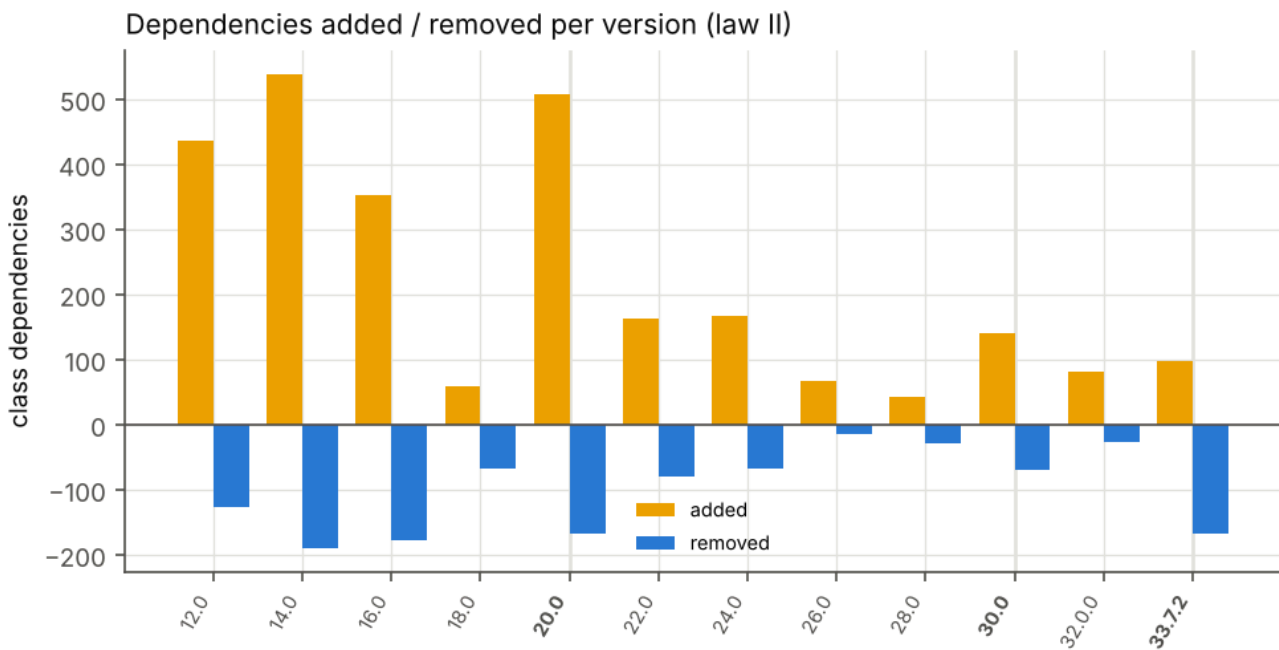


Figure 16 - Dependencies added and removed between sampled versions.

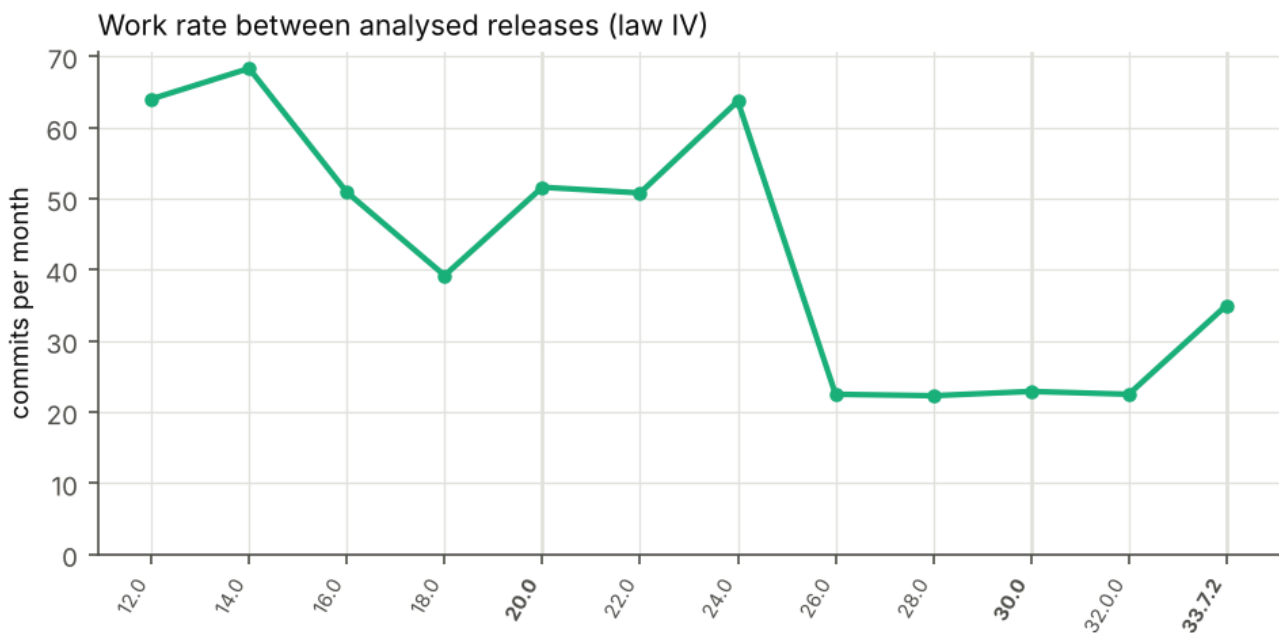


Figure 17 - Commits per month between the sampled releases.

- **Law II - increasing complexity.** As in the lecture's Eclipse example, Figure 16 separates added and removed dependencies. In 10 of the 12 intervals Guava **adds more dependencies than it removes** (e.g. +538/-190 in 14.0, +508/-167 in 20.0). Only 18.0 (+59/-68) and 33.7.2 (+97/-168) are small net reductions. There are no large clean-up releases, so complexity grows almost unchecked (3.2).
- **Law IV - conservation of organisational stability does not hold over 15 years.** The work rate falls from 64-68 commits/month (2011-2013) to 39-64 (2013-2018) and ≈ 22 (2018-2023), then rises again to 35 (2023-2026). This fits a maturing library whose work shifts to maintenance. (Until 18.0 Guava was mirrored from Google's internal repository, so only 4-7 author identities appear in that period.)

- **Law VIII - feedback system.** Feedback reaches Guava through issues, @Beta APIs that may change after user feedback, deprecation cycles before removal, and the parallel JRE/Android flavours.
- Guava is an **E-type** system: it has to follow the Java platform (Java 8 support between 20.0 and 24.0, JDK changes), so the laws apply. Its strong compatibility promise helps explain why the signs of laws II and VII are visible: the public structure is costly to refactor.

Table 18 – Dependencies added and removed between sampled versions.

Version	Dependencies	Added	Removed
12.0	1597	436	128
14.0	1945	538	190
16.0	2121	353	177
18.0	2112	59	68
20.0	2453	508	167
22.0	2534	162	81
24.0	2633	166	67
26.0	2686	67	14
28.0	2699	42	29
30.0	2769	140	70
32.0.0	2821	80	28
33.7.2	2750	97	168

4. Architecture recovery with AI (latest version only: 33.7.2)

Tool and protocol. The AI tool is **Claude (Anthropic)**. We used four prompts of increasing context, given literally in Appendix B:

- prompts: guava/ai/prompts.md;
- inputs: guava/ai/inputs/;
- verbatim responses: guava/ai/responses/.

Prompt	Information given
P1	Name, version and URL only
P2	README (tag v33.7.2) + the 17 packages with class counts and sample class names
P3	P2 + the complete list of 554 classes per package + the package dependency graph (50 weighted edges from DependencyExtractor). No class-level dependencies. Because <code>common.collect</code> holds 38 % of the classes, the AI had to map every class (not only every package) to exactly one component, as ordered regex rules in JSON
P4	Follow-up in the P3 conversation with the cohesion, coupling, MQ and MoJoFM that we measured for the P3 architecture, next to ACDC, k-means and the packages

Provenance.

- Each prompt was answered by a **newly started Claude sub-agent**. It had no conversation history and no access to our results, and was told to read only the attached

files (P1: none). Its tool calls were checked: each agent read exactly its attached files and nothing else.

- The agents ran in the same environment as the analysis, so this isolation rests on instruction, not on technical separation.
- The Claude session that coordinated the work also wrote the analysis pipeline. It chose the prompts and inputs and measured the answers afterwards, but did not edit them.
- The material is kept separate:
 - what the AI **received**: inputs/;
 - what it **returned**: responses/, and ai_architecture_P3.json copied unchanged from the P3 answer;
 - what **we measured** afterwards: results/ai_*.csv, Tables 10–12 and 14.
- For a fully independent comparison the prompts should be repeated in a fresh chat (or with another tool, e.g. ChatGPT).

What the AI produced.

- **P1 (no context).** The AI said that it could not verify that 33.7.2 exists, and that its knowledge reaches about 33.4–33.5. It gave a package-level layered view (base → primitives/math → collect → services → event bus/cache), the build artifacts (-jre/-android, failureaccess, testlib) and a long list of design patterns (builders, immutable objects, forwarding decorators, template methods, ...). It listed its uncertain points, e.g. the strength of net → hash and possible cycles between collect and primitives/math.
 - The data confirm eventbus → cache and eventbus → reflect (3 and 1 dependencies) and net → hash (3).
 - They show **no** package cycle.

Guava is old, stable and well known, so its general knowledge matches the code well.

- **P2 (README + packages).**
 - 12 components in 4 layers.
 - All dependencies clearly marked as **inferred** from names, not measured.
 - 12 possible smells, first of all common.collect as a god package (209 of 554 classes). This is the god component of our smell analysis (3.8).
 - Other smells: the escaping code spread over 4 packages, AbstractIterator duplicated in base and collect, a public internal package, and the README examples still naming 33.7.0 (correct: the README at tag v33.7.2 does).
- **P3 (all classes + package graph).**
 - **Components.** 14 components. collect is split into *immutable*, *types* (multimap, multiset, bimap, table, range) and *utilities*, and util.concurrent into *futures* and *services/synchronisation*. escape/html/xml/net/publicsuffix and primitives/math are merged.
 - **Dependencies and cycles.** It aggregated the package graph into component dependencies and said that the dependencies *inside* its splits are not in the data. It correctly found the package graph acyclic and predicted, as an estimate, cycles between its collect and util.concurrent sub-components. Our measurement confirms them (Table 12: Types ⇌ Utilities 176/23, Immutable ⇌ Utilities 145/39, Immutable ⇌ Types 62/11, ServicesSync ⇌ Futures 27/4).
 - **Hub classes.** Its estimate (Preconditions, ImmutableList/Set/Map, Iterables/Iterators, Lists/Maps/Sets, Ordering, ...) corresponds closely to JNode's most significant classes (3.3).
 - **Class counts.** Its counts per component are close to the measured ones (Table 10): e.g. 79 vs 80 for the utilities, because it counted all 554 classes, not only the 549 connected ones.

• **P4 (metrics feedback).**

- It explained that MQ favours small clusters (cohesion divides by N_i^2), so the gap to ACDC is mostly a matter of granularity.
- At similar granularity its architecture is slightly worse than the packages (MQ 0.078 vs 0.084) despite a lower coupling (0.0035 vs 0.0042).
- It attributed this to its two merges with few internal links and to splits made by concept rather than by dependency. Both diagnoses are confirmed by the measured data: PrimitivesMath has cohesion 0.037, and the collect splits cut through the heaviest dependencies (Table 12).
- It proposed ~16 components: separate Primitives/Math and Escaping/Net, split collect by type family, and merge util.concurrent back. It labelled the expected effect as an estimate.

Table 10 – Components of the AI architecture (P3), measured (connected classes).

Component	Classes
Collect.Utilities	80
Collect.Types	77
Concurrent.Futures	55
Collect.Immutable	52
Graph	52
Base	47
PrimitivesMath	37
IO	34
Hash	31
Concurrent.ServicesSync	23
EscapeNet	21
Cache	19
Reflect	13
EventBus	8

Table 11 – AI (P3) architecture vs the other architectures of 33.7.2.

Architecture (33.7.2)	k	Largest	Cohesion	Coupling	MQ
AI (P3), full	14	80	0.08159	0.003517	0.07807
AI (P3), no noise	14	53	0.0696	0.00082	0.06878
A1 ACDC	90	47	0.25423	0.004754	0.24947
A2 ACDC–noise	139	25	0.15111	0.000844	0.15026
A3 k-means	69	40	0.17635	0.009588	0.16676
A4 k-means–noise	68	15	0.16407	0.000925	0.16315
Packages	17	209	0.08848	0.004248	0.08423

Table 12 – Strongest dependencies between the AI components (measured, class level).

From	To	Class dependencies
Collect.Types	Collect.Utilities	176
Collect.Immutable	Collect.Utilities	145
Collect.Utilities	Base	73
Collect.Types	Base	72
Collect.Immutable	Collect.Types	62
Graph	Base	45
IO	Base	43
Concurrent.Futures	Base	40
PrimitivesMath	Base	40
Collect.Immutable	Base	39
Collect.Utilities	Collect.Immutable	39
Graph	Collect.Utilities	37
Cache	Base	27
Concurrent.ServicesSync	Concurrent.Futures	27
Concurrent.ServicesSync	Base	27

5. Reflection

5.1 Comparing the architectures of the latest version

- **Granularity.** AI: 14 components; packages: 17; k-means: 68–69; ACDC: 90 (139 without noise). The AI and the packages are at the level of a component diagram. The algorithmic architectures are fine-grained groups of collaborating classes.
- **Quality (MQ).** On the full graph:

Architecture	MQ
ACDC	0.249
k-means	0.167
Packages	0.084
AI	0.078

- **AI vs packages.** With a similar number of components, the AI architecture has the lower coupling but also the lower cohesion. Its `collect` split cuts through dependencies, and its merges join packages that hardly use each other (Table 12, P4).
- **ACDC vs the rest.** The large gap comes from cluster size: MQ divides by N_i^2 , so components of up to 80 classes cannot reach the cohesion of 6-class clusters.
- **Similarity (MoJoFM).**
 - The AI architecture is very close to the packages (94.4 %).
 - Of the algorithms, k-means is closer to the AI architecture (67 %) than ACDC (61–65 %).
 - For a library organised by feature, the developers' packages and an architect's view largely coincide.
- **Different principles.** The algorithms group classes that *use the same things*. For example, ACDC's `Multimaps.ss` gathers the multimap implementations and their utilities, and `Hashing.ss` the hash functions. These are sensible class families, but they cut across

the feature boundaries that Guava's users and maintainers work with. The AI groups by responsibility and layer.

Our assessment.

- **The AI architecture (P3)** is the most useful **top-level view** of Guava. It is close to the documented package structure and explains its layering, hubs and smells, and these are consistent with our measurements (acyclic package graph, god component collect, cycles inside collect). By MQ, however, it is not better than the packages.
- **ACDC** gives the best modularised decomposition by the course's quality measure, and it is the most stable across versions (MoJoFM $\approx 90\%$). It is the best *automatic* method and the right tool to look inside large packages such as `collect` and `util.concurrent`.
- **k-means** is less cohesive and much less stable (MoJoFM $\approx 71\%$).

5.2 Does the AI tool help? How do information and prompts influence the result?

- **For a well-known, stable system the AI is already reasonable without context (P1).** It said openly that it did not know the exact version. For a young, fast-changing system we would expect much more outdated or invented content. Extra information mainly makes the answer more **precise and checkable**. With the class list and the dependency graph (P3) it becomes a complete class mapping with dependency weights.
- **The granularity of the input determines the granularity of the output.**
 - With packages only (P2), the AI cannot see inside `collect`, and all its dependencies are guesses (which it said).
 - With the class names (P3), it split the `god` package using naming conventions (`Immutable*`, `Regular*`, `*Multimap`). Without class-level dependencies, though, it could not see that these splits cut through the densest part of the graph.
- **Feeding back metrics (P4) works.** The AI explained the MQ results correctly and found the weak parts of its own decomposition.
- **Risks.** The AI does not measure. Every structural claim had to be checked against the extracted data. Its estimates (cycles inside its splits, hub classes) were right here, but they were estimates. The isolation of the agents rests on instruction (Section 4).

5.3 Which method for other systems?

1. **For the top-level view, use the AI together with the package structure.** Give the AI the extracted facts, including class-level dependencies if it has to split packages. This is especially effective for libraries and well-known systems.
2. **For evolution, use ACDC.** It has the best MQ, it is deterministic and the most stable across versions, and its clusters show the internal families of large packages.
3. **Treat the removal of widely-used classes with care.** In Guava the omnipresent classes carry $\sim 70\%$ of the dependencies. Removing them isolates ~ 50 classes, which lowers ACDC's MQ and MoJoFM, while k-means gets closer to the packages. Compare two methods (JNode and Bunch) and report how many classes the removal isolates.
4. **Check the tools.** The course's JNode built a wrong dependency graph (2.3). This was only found by comparing its internal graph with `DependencyExtractor`.
5. k-means needs extra decisions (features, k) and is the least stable. It is useful as a second opinion, not as the primary method.

6. Threats to validity

- **Sparse sampling.** Every second major release (gaps of 6 months to 3.3 years) hides short-term changes and lowers the stability values.
- **Binary analysis** of the `-jre jar`: reflection and the Android flavour are not covered. The analysis is static; runtime behaviour is not considered.
- **Tools.**
 - JNode was fixed: two graph defects, precision, and speed. The fixed graph is identical to DependencyExtractor's, but the noise classes are therefore not those of the unmodified course tool (2.3).
 - ACDC is the ARCADE port and needed file-style names (2.3).
- **Populations.** A2/A4 keep the classes that the noise removal isolates, as singletons for ACDC. This choice lowers A2's MQ and MoJoFM (3.4, 3.6). We therefore also report MoJoFM on the linked classes.
- **No expert architecture.** MoJoFM uses the package structure and the AI architecture as references, not an expert-validated decomposition.
- **MQ depends on cluster size.** MQ favours small, dense clusters, punishes singletons, and is barely affected by one very large cluster (3.4). We always read it together with the number and size of the clusters.
- **AI phase.** The answers come from one tool (Claude) and one run each. The sub-agents were isolated by instruction only (Section 4).

References

- Lecture notes EPL484 (E. Constantinou): Lectures 1-2 (Lehman's laws), 5 (Architecture evolution, architectural smells), 6-7 (Architecture recovery: cohesion, coupling, MQ, ACDC, omnipresent classes, MoJoFM).
- Mancoridis S. et al. (1998). *Using Automatic Clustering to Produce High-Level System Organizations of Source Code*. IWPC.
- Tzerpos V., Holt R. C. (2000). *ACDC: An Algorithm for Comprehension-Driven Clustering*. WCRE.
- Constantinou E. et al. (2015). Identification of omnipresent classes (JNode).
- Fontana F. A. et al. (2016). *Automatic detection of instability architectural smells*. ICSME.
- Sas D. et al. (2019). *Investigating instability architectural smells evolution: an exploratory case study*. ICSME.
- Lehman M. M. (1996). *Laws of Software Evolution Revisited*. EWSPT.
- Guava README, user guide (wiki) and Javadoc, v33.7.2.

Appendix A - Full metric tables

Table 5 – Number of clusters (k).

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
10.0	41	72	52	34	8
12.0	51	89	52	58	11
14.0	70	110	52	46	11
16.0	72	122	60	49	15
18.0	74	117	52	49	15
20.0	80	119	52	56	16
22.0	79	121	74	64	16
24.0	88	125	74	60	16
26.0	89	131	59	60	16
28.0	90	131	54	60	16
30.0	96	139	69	56	16
32.0.0	95	136	59	64	16
33.7.2	90	139	69	68	17

Table 7 – MQ.

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
10.0	0.2507	0.1192	0.1855	0.1808	0.0924
12.0	0.2524	0.1227	0.1759	0.1974	0.1208
14.0	0.2545	0.1294	0.1745	0.1791	0.1121
16.0	0.2599	0.1253	0.1791	0.1863	0.1102
18.0	0.2619	0.1314	0.1711	0.1729	0.1080
20.0	0.2527	0.1318	0.1691	0.1700	0.0993
22.0	0.2508	0.1214	0.1663	0.1666	0.0992
24.0	0.2494	0.1338	0.1608	0.1656	0.0957
26.0	0.2456	0.1376	0.1559	0.1694	0.0956
28.0	0.2486	0.1395	0.1522	0.1734	0.0963
30.0	0.2429	0.1412	0.1702	0.1617	0.0947
32.0.0	0.2479	0.1375	0.1691	0.1661	0.0950
33.7.2	0.2495	0.1503	0.1668	0.1631	0.0842

Table 8a – Cohesion.

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
10.0	0.2586	0.1197	0.1937	0.1821	0.0989
12.0	0.2628	0.1231	0.1823	0.1986	0.1259
14.0	0.2613	0.1302	0.1811	0.1803	0.1176
16.0	0.2670	0.1262	0.1871	0.1875	0.1156
18.0	0.2686	0.1321	0.1775	0.1742	0.1133
20.0	0.2594	0.1325	0.1746	0.1712	0.1040
22.0	0.2566	0.1220	0.1741	0.1676	0.1038
24.0	0.2556	0.1344	0.1672	0.1669	0.1005
26.0	0.2511	0.1382	0.1607	0.1706	0.1005
28.0	0.2541	0.1403	0.1572	0.1747	0.1011
30.0	0.2482	0.1420	0.1760	0.1626	0.0994
32.0.0	0.2535	0.1382	0.1748	0.1672	0.0998
33.7.2	0.2542	0.1511	0.1764	0.1641	0.0885

Table 8b – Coupling.

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
10.0	0.00787	0.00045	0.00817	0.00134	0.00649
12.0	0.01043	0.00041	0.00640	0.00122	0.00510
14.0	0.00684	0.00081	0.00655	0.00121	0.00551
16.0	0.00706	0.00089	0.00802	0.00125	0.00539
18.0	0.00664	0.00076	0.00637	0.00126	0.00533
20.0	0.00666	0.00069	0.00540	0.00123	0.00475
22.0	0.00588	0.00060	0.00775	0.00105	0.00468
24.0	0.00628	0.00066	0.00640	0.00124	0.00481
26.0	0.00549	0.00066	0.00483	0.00112	0.00481
28.0	0.00553	0.00075	0.00498	0.00122	0.00482
30.0	0.00540	0.00076	0.00581	0.00093	0.00473
32.0.0	0.00560	0.00074	0.00573	0.00106	0.00473
33.7.2	0.00475	0.00084	0.00959	0.00093	0.00425

Table 9 – Package and class changes between sampled versions.

Version	Packages	Pkgs added	Pkgs removed	Classes added	Classes removed
12.0	13	3	0	63	9
14.0	13	0	0	58	23
16.0	17	4	0	50	14
18.0	17	0	0	8	4
20.0	18	1	0	76	16
22.0	18	0	0	20	5
24.0	18	0	0	24	4
26.0	18	0	0	7	0
28.0	18	0	0	2	4
30.0	18	0	0	19	5
32.0.0	18	0	0	6	1
33.7.2	18	0	0	14	18

Table 19 – Commits between the sampled releases.

From	To	Days	Commits	Authors	Commits/month
10.0	12.0	216	454	5	64.0
12.0	14.0	301	675	7	68.3
14.0	16.0	326	545	6	50.9
16.0	18.0	220	283	4	39.2
18.0	20.0	795	1348	143	51.6
20.0	22.0	206	344	69	50.8
22.0	24.0	255	534	79	63.7
24.0	26.0	181	134	49	22.5
26.0	28.0	314	230	75	22.3
28.0	30.0	493	371	119	22.9
30.0	32.0.0	952	705	70	22.5
32.0.0	33.7.2	1222	1404	66	35.0

Appendix B - The AI prompts (literal)

The inputs of each prompt are listed in `guava/ai/prompts.md`, and the answers are in `guava/ai/responses/`.

P1 - zero context

You are an experienced software architect. Recover the software architecture of Google Guava, version 33.7.2 (<https://github.com/google/guava>).

1. Identify the main architectural components, their responsibilities and the most important classes or interfaces of each.
2. Describe the dependencies between the components.
3. Give the architecture as a layered component diagram (text/ASCII).
4. State the architectural styles and design patterns you recognise.

Be explicit about what you are unsure of.

P2 - documentation + package structure

You are an experienced software architect. I attach (a) the README of Guava 33.7.2 and (b) the list of its Java packages with the number of top-level classes and some class names.

Using ONLY this information:

1. Group the packages into architectural components and give each a name and a responsibility.
2. Organise the components in layers and describe the allowed dependencies.
3. Draw a layered component diagram (text/ASCII).
4. Point out anything that looks like an architectural smell.

P3 - code-level facts: all classes and the package dependency graph

You are an experienced software architect performing architecture recovery. I attach for Guava 33.7.2: the README, the packages, the complete list of top-level classes per package and the package dependency graph extracted from the bytecode ("a -> b : n" = n class-level dependencies from package a to b; prefix com.google. omitted).

1. Recover the architecture: define between 8 and 15 components. Every class must belong to exactly one component; large packages may be split. Give the mapping as an ORDERED list of (component, Java regular expression over "package.ClassName") rules in JSON where the first matching rule wins.
2. For each component give its responsibility and key classes.
3. Describe the dependencies between your components (strongest ones, cycles?) and draw a layered component diagram.
4. Which classes/packages are hubs used almost everywhere?
5. Assess the modularity of the library: strong points and weak points.

P4 - follow-up with the measured quality metrics

I mapped every class to your components with your rules and computed the metrics below for Guava 33.7.2 (full system: 549 classes that have at least one dependency, 2,750 class-level dependencies). Cohesion = mean intra-connectivity $A_i = \mu_i / N_i^2$; coupling = mean inter-connectivity $E_{ij} = \text{eps}_{ij} / (2 N_i N_j)$; MQ = mean cohesion - mean coupling; MoJoFM to the package structure. They are shown next to the ACDC and k-means architectures and the packages of the same version.

Architecture	#clusters	Largest	Cohesion	Coupling	MQ	MoJoFM to packages
AI (your P3 architecture)	14	80	0.0816	0.00352	0.078	94.38 %
A1 ACDC	90	47	0.2542	0.00475	0.249	70.79 %
A3 k-means	69	40	0.1764	0.00959	0.167	73.6 %
Packages	17	209	0.0885	0.00425	0.084	-

MoJoFM to your architecture: A1 64.86 %, A3 67.48 %, packages 70.47 %.

Explain the differences. Would you change your decomposition, and if so how, without losing its meaning for a developer?